

A4S: CO-DESIGNING SCALING AND PLACEMENT IN STREAM PROCESSING SYSTEMS
WITH MEMORY-PARALLELISM CURVES

by

Kexiang Wang

A thesis submitted in conformity with the requirements
for the degree of Master of Applied Science
Department of Electrical and Computer Engineering
University of Toronto

© Copyright 2023 by Kexiang Wang

Kexiang Wang
Master of Applied Science

Department of Electrical and Computer Engineering
University of Toronto
2023

Abstract

A significant challenge in stream processing systems is adapting to workload changes while maintaining throughput and end-to-end latency requirements. Traditional auto-scaling solutions perform CPU scaling by adjusting operator parallelism, which works well for stateless operators. However, these solutions perform poorly for stateful operators because they do not consider the impact of operator memory size and IO operations. Another challenge in existing solutions is that operator scaling and task placement are performed independently, which leads to sub-optimal resource allocation.

This thesis presents A4S, a co-designed scaling and placement strategy for stream processing systems that works well for both stateless and stateful operators. The key insight in A4S is that, for achieving a certain throughput, the memory and parallelism requirements of stateful operators are correlated. This insight allows trading memory and parallelism, leading to better scaling and placement. We formalize this correlation with a memory-parallelism curve, which shows the minimum operator memory size needed for a given level of parallelism to achieve a target throughput. These curves provide a set of parallelism choices for operator scaling. Then, task placement chooses a level of operator parallelism and memory size that ensures a target throughput, acceptable end-to-end latency and efficient resource utilization.

We show that memory-parallelism curves can be derived from the miss-rate curves of stateful operators. For effective scaling, we need a miss-rate curve estimate of the current workload pattern. Thus, we propose an online method for generating miss-rate curves that has low overhead and provides reasonable accuracy.

We have implemented A4S in the Risingwave streaming database. Our evaluation shows that A4S provides more accurate scaling and resource-efficient placement compared to state-of-the-art solutions.

Contents

1	Introduction	6
2	Related Work and Motivation	9
2.1	Streaming Systems	9
2.1.1	Scaling Streaming Systems	11
2.1.2	Challenge With Scaling Stateful Operators	11
2.2	Miss-Rate Curves	12
2.2.1	Generating Miss-Rate Curves	12
3	A4S Design	14
3.1	Workload Assumptions	14
3.2	Memory-Parallelism Curves	14
3.2.1	The Relation Between Memory and Parallelism	15
3.2.2	Understanding Memory-Parallelism Curves	15
3.2.3	Range of Memory-Parallelism Curves	17
3.3	Co-Designing Scaling and Placement	18
3.4	Quickmrc	21
3.4.1	Generating Stack Distance	22
3.4.2	Generating Miss-Rate Curve	25
4	Implementation and Deployment	27
5	Evaluation	31
5.1	Methodology	31
5.1.1	Experimental Environment	31
5.1.2	Workloads	31
5.2	A4S Scaling and Placement Performance	34
5.2.1	Setup	34
5.2.2	Results	34
5.3	A4S Overhead	36
5.4	Performance and Accuracy of Quickmrc	39
6	Conclusions	41
7	Future Work	42

List of Tables

3.1	Calculating the unscaled miss rate curve using merged stack distances. $h(d)$ is the number of cache hits with stack distance d . $HR(n)$ and $MR(n)$ are hit rate and miss rate with cache size n	25
5.1	Queries and their characteristics.	32
5.2	The output of A4S and DS2 scaling policies. <i>Parallelism</i> refers to the parallelism level of the bottleneck operator before and after scaling. <i>WorkerNum</i> refers to the number of workers (compute nodes) required for the query before and after scaling.	35
5.3	Memory-parallelism curves generated by A4S.	35

List of Figures

2.1	An example a logical directed acyclic graph (DAG) and its corresponding physical graph for a streaming job.	10
2.2	Stateful operators use task-local caches to reduce expensive storage accesses.	10
2.3	Throughput of a memory-intensive query when scaling with DS2. The scaling target is 150K records/s, but the query throughput reaches 300K records/s after scaling.	12
3.1	An example of memory-parallelism curves. MPC_1 and MPC_2 are two memory-parallelism curves for a join operator with two target throughputs, 75000 and 150000 records/s.	16
3.2	Estimating stack distance based on bucket sizes.	22
3.3	Estimating the miss rate curve of an operator from the stack distance streams of its two tasks.	26
4.1	The Risingwave Architecture with the A4S Controller.	28
5.1	Miss-rate curves for the Q4, Q18 and Q20 NEXMark queries.	32
5.2	Zoomed in miss-rate curves for the Q4, Q18 and Q20 NEXMark queries.	33
5.3	Memory-parallelism curves generated by A4S.	36
5.4	The top row shows query throughput before and after scaling with the DS2 and A4S scaling policy. The middle and bottom rows show the average, median, and P99 latency before and after scaling with A4S and DS2 respectively.	37
5.5	A4S throughput with different ghost cache sizes. For example, $2\times$ means the ghost cache has twice the number of items in the LRU cache.	38
5.6	A4S latency with different ghost cache sizes.	38
5.7	Throughput of an LRU cache using Quickmrc with different ghost cache sizes.	39
5.8	Miss rate curves with LRU buckets, sampling and histogram buckets in Quickmrc.	40

Chapter 1

Introduction

A stream processing system enables real-time processing and analysis of data streams [8, 9, 15, 42, 53, 68]. To use these systems, developers create streaming jobs and connect them to data source systems, such as Kafka [67] or AWS Kinesis [1]. Then the jobs consume data from these sources, incrementally process the data, update query results, and store them to sink systems, such as AWS S3 [2], offering users up-to-date information.

Stream processing systems provide auto-scaling features that enable jobs to adapt to changing workloads to meet their performance requirements, such as a target throughput and latency [9, 25, 26, 36, 61]. A streaming job consists of a set of interconnected operators. Auto-scaling involves *operator scaling* and *operator placement*. Each operator is assigned a certain number of tasks that defines its level of parallelism, i.e., the number of (single-threaded) tasks executing the operator’s logic. Operator scaling determines the parallelism needed for each operator so that the job can sustain its target throughput. Each task runs on a worker, such as a container. Operator placement assigns all the tasks of the job to workers efficiently, e.g., minimizes the number of workers while meeting the resource requirements of tasks and resource capacities of the workers.

Current systems use heuristic or predictive operator scaling policies. Heuristic policies use congestion and back-pressure signals to locate bottleneck operators [25]. They are usually expressed as a set of rules that use predefined thresholds and conditions. While these policies are simple and intuitive, they can cause temporary over- or under-provisioning and long convergence times [36]. Predictive policies take advantage of the streaming queries’ DAG model [22], and they use operator performance metrics, such as true rates [36], to perform more accurate operator scaling. However, current policies primarily focus on CPU scaling and do not work well for stateful operators, such as join and aggregation, which require significant memory for caching intermediate states to reduce expensive IO operations. For example, they increase operator parallelism by allocating additional tasks to achieve a higher target throughput, but this scaling decision allocates more memory to the operator as well. Assuming the same data access pattern, the additional memory reduces cache misses, leading to a super-linear increase in achievable operator throughput or inefficient resource allocation for the target throughput. Many streaming systems [15, 50, 68] now support disaggregated remote storage, such as AWS S3 [2], with higher IO latencies (more than 20 ms for 4KB read) than local storage (5-10 ms on HDD and 0.1-0.5 ms on SSD for 4KB read), which increases the impact of cache misses.

Current systems perform operator placement using integer linear program (ILP) solvers [16, 46, 57]. Their placement decisions for stateful operators are based on inaccurate operator scaling, as described above, and they also do not take the memory needs of these operators into account, which limits their flexibility and

leads to inefficient resource allocation.

This thesis presents A4S, an auto-scaling strategy that enables streaming jobs to meet their performance requirements, such as a target throughput and latency, while utilizing resources efficiently. A4S is specifically designed for stateful operators in streaming systems. Our key insight is that, to achieve a given throughput, the memory requirements and parallelism of stateful operators are correlated, which enables trading one for the other. For example, we can increase the throughput of a memory-intensive operator by either 1) allocating more memory without increasing parallelism when there is limited CPU available, or 2) increasing parallelism without allocating more memory when there is limited memory available. This flexibility allows for making more efficient scaling and placement decisions.

We formalize the correlation between memory requirements and parallelism of stateful operators with a memory-parallelism curve, which shows the minimum memory requirements of an operator to achieve a given throughput within a range of minimum and maximum parallelism levels. The minimum parallelism is chosen, assuming that the operator has enough memory to run without performing any IO. As parallelism increases, the throughput needed per task decreases, so the amount of IO a task is allowed to perform increases, which increases end-to-end latency. The maximum parallelism is chosen to ensure that the job can maintain its target latency.

While memory-parallelism curves enable more flexibility with operator scaling and placement decisions, they also significantly increase the space of choices available for task placement. We simplify and co-design scaling and placement with a greedy strategy that starts with the minimal parallelism of operators, which provides the best end-to-end latency while requiring the most amount of operator memory. Then we progressively increase operator parallelism, which reduces their memory requirements, until the operators can be allocated on the current set of workers. If this fails, even when maximum parallelism is reached, we progressively allocate more workers.

A4S requires memory-parallelism curves during scaling and placement operations. To generate these curves, we need to know how the operator cache size affects the number of IO operations and thus the operator throughput, which we show can be estimated using operator miss-rate curves. For effective scaling, we need to generate miss-rate curves online to estimate the current workload pattern. In streaming systems, some overestimation of memory requirements is acceptable because it helps tolerate bursty streams. Thus, our goal is to generate miss-rate curves with minimal overhead and reasonable accuracy.

We propose an online method for generating miss-rate curves, called Quickmrc, that builds on two efficient approximation methods, grouping accesses [58] and sampling accesses [66]. Quickmrc groups or buckets the entries in the cache, similar to MIMIR [58], to reduce the time needed to calculate stack distance and uses a ghost cache for estimating the miss-rate curve beyond the current cache size. To reduce overhead further, Quickmrc uses the SHARDS [66] spatial sampling strategy. This combined approach has low overhead and provides reasonable accuracy for typical stateful operators.

We have implemented A4S as a separate component in the Risingwave streaming database. Our evaluation compares A4S with recent state-of-the-art scaling and placement solutions. We show that A4S can achieve better scaling accuracy and more efficient placement. We also compare Quickmrc with recent solutions and show its benefits.

We summarize our contributions as follows:

- We analyze the limitations of state-of-the-art scaling and placement solutions.
- We introduce memory-parallelism curves for stateful operators, which capture the correlation between

operator memory requirements and operator parallelism for achieving a given throughput.

- We introduce A4S, a novel auto-scaling solution for streaming systems that uses operators' memory-parallelism curves to enable more accurate scaling and efficient placement.
- We present Quickmrc, an online method for generating miss-rate curves that has good performance and reasonable accuracy.

Chapter 2

Related Work and Motivation

In this chapter, we provide background on scaling methods in streaming systems and their limitations when scaling stateful operators to motivate our approach based on memory-parallelism curves. Next, we describe existing work on generating miss-rate curves to motivate our Quickmrc scheme.

2.1 Streaming Systems

Modern streaming processing systems provide familiar SQL-like query language interfaces for writing complex streaming analytics jobs [9, 15, 40, 41, 50, 68, 73]. A streaming job is parsed and converted to a logical dataflow graph, usually a directed acyclic graph (DAG), as shown on the left side of Figure 2.1. The vertices of the graph are functions or operations, such as Aggregation, Filter, and Join, and they are called operators in streaming systems. These operators are interconnected with edges that serve as data channels between operators. Operators process input events from upstream operators and emit output events to downstream operators along the edges.

A streaming system deploys a logical dataflow graph by translating it into a physical dataflow graph at runtime, as shown on the right side of Figure 2.1. Each operator is mapped to one or more (single-threaded) tasks that execute in parallel, which enables scaling operator throughput to meet the job’s performance requirements. The number of tasks associated with an operator is called the operator’s parallelism.

The input events of an operator are partitioned by input keys into multiple partitions and each partition is handled by one task. This partitioning ensures that a given key is always processed by the same task, which avoids the need to synchronize the processing of events across tasks. Typically, the input is hash-partitioned for easier load balancing. As shown in Figure 2.1, the tasks of different operators are connected by edges that indicate input/output-relationships between the data partitions.

In a distributed streaming system, tasks are deployed on one or more workers. A worker could be a process running in a container, a virtual machine, or a physical machine. The tasks utilize the CPU and memory resources of workers to execute their operations and communicate with each other through local buffers or networks.

An operator can be either stateless or stateful. A stateful operator keeps the state from previously processed data records (e.g., count of records processed) and uses the state to process the current record. Stateful operators such as join and aggregation require significant states, and thus they store this state on storage and use an in-memory cache for better performance, as shown in Figure 2.2. The caches are task-local (thread-local) because the data partitions are disjoint, and so a task does not need to access the data at other tasks.

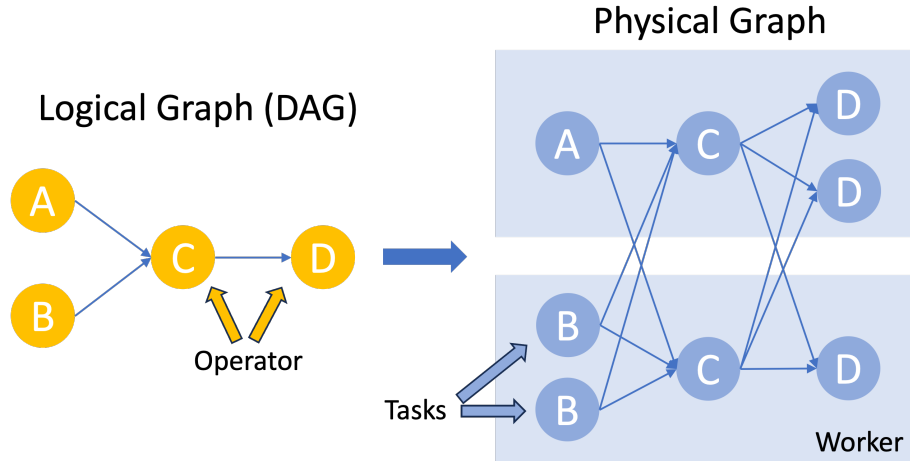


Figure 2.1: An example a logical directed acyclic graph (DAG) and its corresponding physical graph for a streaming job.

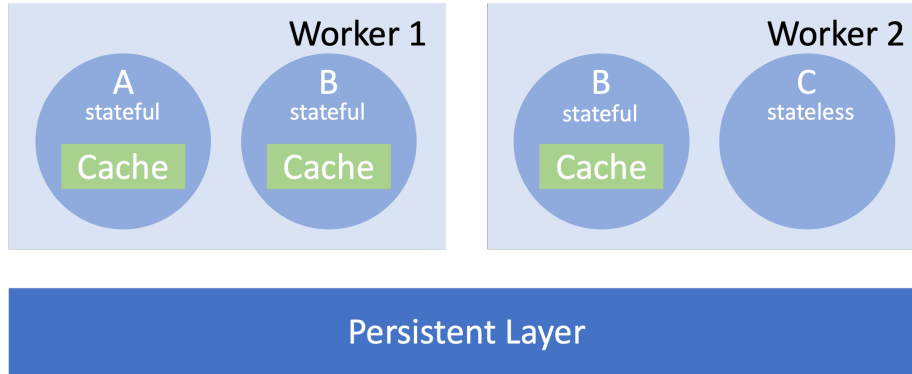


Figure 2.2: Stateful operators use task-local caches to reduce expensive storage accesses.

When handling a record, a task looks up the corresponding state for the record in the cache. On a cache miss, the task fetches this state from the persistent layer and waits for the completion of the (slow) IO operation. Then, the task updates the state in the cache and writes this state to the persistent layer asynchronously. While stateless operators primarily require CPU resources, stateful operators require both CPU resources and memory resources for their caches. However, even with caching, when key accesses have poor locality, IO can become a bottleneck for stateful operators. While streaming systems require memory for several components such as the write and network buffers, operator caches have the most impact on the performance of stateful operators. Thus, in the following sections, when discussing operator memory requirements, we mainly focus on the operator caches.

In recent years, streaming systems like Risingwave have started adopting a disaggregated architecture, with a stateless compute layer and a stateful storage layer [50, 68]. The storage layer is implemented on a cloud storage service, such as AWS S3 [2] and Azure Blob Storage [3], that provides good scalability but has high access latencies (> 20 ms) [65], which increases the impact of IO on stateful operators.

2.1.1 Scaling Streaming Systems

In the real world, streaming systems are expected to serve dynamic and frequently changing workloads while providing various service-level guarantees such as target throughput, latency, and low downtime. They meet these requirements of streaming jobs by adding more hardware resources to clusters and then reconfiguring their operators and migrating their states.

For scaling performance, one can either add more machines to the cluster (scale out) or use more resources (e.g., CPU) on existing machines (scale up). Traditional streaming systems such as Flink [15], Heron [42], Risingwave [68] and Millwheel [8] are built for scaling out. They persist and manage data on disk and provide strong exactly-once guarantees. On the other hand, some systems are designed for scaling up, often using a single powerful machine with multiple cores [28, 33, 34, 40, 51]. Scale-up systems usually perform better [40] since they do not need to use the network or manage storage, but they lack fault tolerance and semantic guarantees. These systems demand expensive hardware and are limited by the size of the machine. Hence, scale-out systems are more popular in production, and this thesis focuses on enhancing them.

Many systems provide manual reconfiguration mechanisms [23, 31, 48, 71], but manual reconfiguration requires extensive operational experience to decide when and how much to scale and place operators, making it tedious and time-consuming [25]. Recent work has proposed several auto-scaling methods that target different objectives, including end-to-end latency [17, 24, 29, 61, 69], data skew [17, 39, 56, 74], multi-tenant deployment [37], WAN awareness [35] and resource utilization [46, 57].

This thesis focuses on meeting the throughput requirements of a streaming job. One approach to achieving a target throughput is monitoring system metrics such as CPU utilization and making scaling decisions based on predefined rules that use thresholds, e.g., *if (CPU utilization > 60% or back-pressure) $\Rightarrow$ scale out* [19, 20, 25, 27, 30]. However, these heuristic policies fail to provide accurate scaling [36].

Another scaling approach is based on monitoring per-operator metrics and utilizing the DAG model of the streaming job to predict the operator configuration needed to achieve the target throughput. For example, DS2 is a state-of-the-art scaling solution [36] that monitors operator throughput and CPU usage to derive the operator’s true rate (operator throughput, assuming that the task does not wait for upstream or downstream operators). It also uses the job’s DAG model to predict the target throughput of each operator needed to meet the overall target throughput of the job. Then, DS2 scales operators based on their true rate and target throughput. Essentially, DS2 uses the relation between the operator’s current CPU usage and throughput to predict the CPU needed and, thus, the level of parallelism or the number of tasks needed to achieve the target throughput. Then, the operator’s tasks are placed on workers using various strategies.

2.1.2 Challenge With Scaling Stateful Operators

While DS2 works well for stateless operators, it makes inaccurate scaling predictions for stateful operators. The reason is that the true rates of operators are based on CPU usage and do not account for the memory allocated to stateful operators and the cost of IO operations. Figure 2.3 shows the throughput of the NEXMark Q4 query, which includes a join operator, processing events with modified timestamps so that IO operations take about 60% of the execution time (see Chapter 5 for more details). For this query, DS2 over-provisions the join operator. While the scaling target is 150K records/second, the throughput achievable after scaling is about 300K records/second, leading to approximately 100% overallocation. In this case, DS2 scales the level of parallelism and the overall memory allocated to the operator (i.e., the sum of the memory allocated to all tasks), leading to a lower cache miss rate and higher throughput per task. As a result, the operator’s

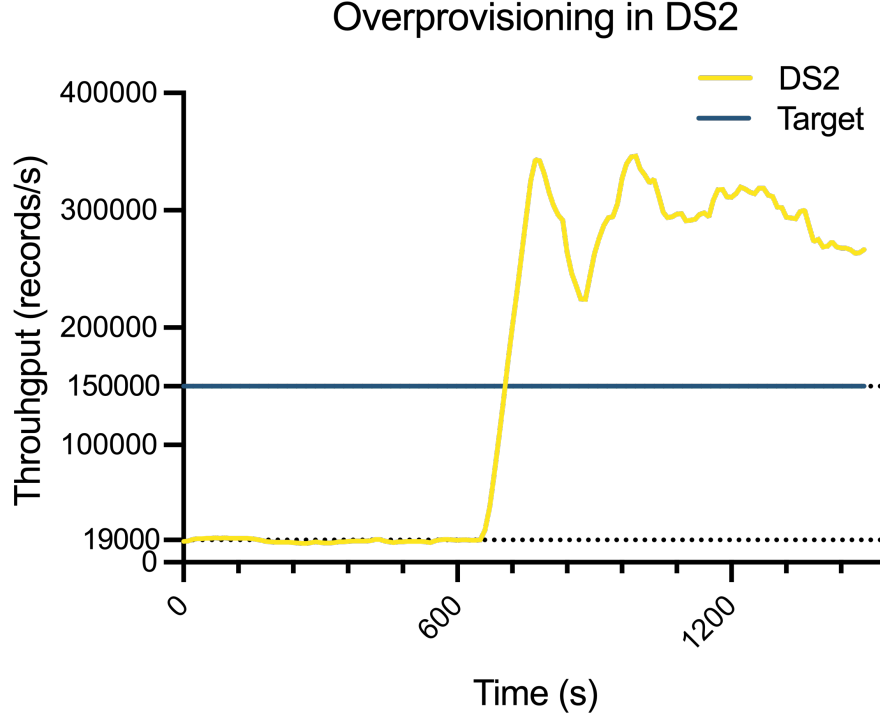


Figure 2.3: Throughput of a memory-intensive query when scaling with DS2. The scaling target is 150K records/s, but the query throughput reaches 300K records/s after scaling.

throughput scales super-linearly with parallelism, leading to over-provisioning, which wastes resources.

The root cause of this issue is that the memory requirements and parallelism of stateful operators are correlated, but existing work scales operators based on CPU usage only, ignoring the memory allocated to operators. We use memory-parallelism curves to scale operators, which enables both more accurate scaling and efficient resource utilization.

2.2 Miss-Rate Curves

Recall that memory-parallelism curves show the minimum operator memory needed to achieve a given throughput at different levels of parallelism. To generate these curves, we need to know how the cache size affects the number of IO operations and thus the operator throughput. This relation can be determined using miss-rate curves that show the cache miss rate for different cache sizes.

2.2.1 Generating Miss-Rate Curves

Most work on generating miss-rate curves has focused on the commonly used Least Recently Used (LRU) cache replacement policy. In an LRU implementation, cached items are stored in a stack, ranked by recency. When an item is accessed, it is extracted and moved to the top of the stack. Its depth from the stack top before it is moved is called the stack (or reuse) distance for this access. This distance is the number of other unique items accessed since the previous access to this item. The stack distance is used to determine whether the access would result in a cache hit or a miss for a given memory size. If the memory size is larger than

the stack distance, the access is a hit, and miss otherwise. The miss-rate curve is generated by using a stack distance histogram.

The main cost associated with generating miss-rate curves is the time and space needed to determine stack distance. Assume an access trace of length N containing M unique items. The original stack distance calculation algorithm by Mattson et al. [49] required a linear traversal of the LRU stack on each access. This approach is expensive, with an asymptotic time complexity of $O(N * M)$ [75]. Bennett and Kruskal [12] replace the LRU stack with a partial-sum tree, allowing stack distance calculation in $O(N * \log(N))$ time. Search trees [55] and Interval trees [10] provide further improvements by tracking only the most recent accesses to each item, allowing stack distance calculation in $O(N * \log(M))$ time. These approaches store all the unique items, and so their space complexity is $O(M)$.

The previous approaches generate exact miss-rate curves. Given the importance of these curves, there has been much effort in generating approximate miss-rate curves more efficiently. Kim et al. [38], MIMIR [58] and Cliffhanger [21] group the M stack locations into B buckets, where B is a small number (e.g., 128). Bucketing lowers the stack distance calculation time to $O(N * B)$. Kim et al. [38] use powers-of-two bucket sizes. On accesses, they track bucket boundaries and adjust the stack distance of items that cross these boundaries. MIMIR [58] uses variable-sized buckets. On an access, an item is moved from its current bucket to the most recent bucket. To balance the number of items across buckets, MIMIR limits the maximum number of items in a bucket and uses an aging policy that moves items between buckets periodically.

Sampling is another approximation scheme that reduces cost by only calculating the stack distance for the sampled accesses. These accesses can be sampled temporally, e.g., sample 1 in every N access [13, 14], or sample for a short period and then disable sampling for the next period [63, 75]. Simply sampling 1 in N accesses can miss infrequently accessed items and thus leads to a bias against longer stack distances [60, 76]. SHARDS [52, 66] proposes spatial sampling, i.e., sampling a fixed set of items. This approach reduces bias because the sample can contain both frequently or rarely accessed items. SHARDS shows that sampling at a fixed rate of 1 in 100-1000 accesses can still provide accurate miss-rate curves while reducing computation costs by 75-128x. However, this sampling still introduces inaccuracy when the sampling rate is low or the data is highly skewed. Thus, Carra et al. [18] use exact miss-rate calculation for small cache sizes while relying on sampling for large sizes to improve accuracy when data is skewed. KRR [72] is a probabilistic stack algorithm for efficiently calculating stack distances for the non-stack K-LRU replacement algorithm.

The bucketing and sampling approaches described above reduce computation time but still have a space complexity of $O(M)$. Counter Stacks [70] reduces space complexity by maintaining a list of counters, with each counter recording the number of unique items accessed since it was instantiated. The stack distance is then estimated by computing intra- and inter-counter changes. Counter Stacks uses various approximation methods, including probabilistic HyperLogLog counters, and downsampling and pruning these counters, and requires roughly $O(\log M)$ space. SHARDS [52, 66] also proposes an adaptive-rate spatial sampling method that requires $O(1)$ space. Its memory usage is constant, but its CPU requirements are roughly 3-4 times its fixed-rate sampling method. This overhead is caused by the non-trivial costs associated with dynamically adapting the sampling rate, including handling evicted samples and updating the sample set.

Our online Quickmrc scheme calculates stack distance using a combination of bucketing and sampling. Bucketing has low computation costs because the bucketed stack distance can be calculated efficiently, and we use fixed-rate sampling, which is efficient. The main overhead in our scheme is the space required for the ghost cache, which impacts the memory available for the stateful operators.

Chapter 3

A4S Design

This chapter describes the design of the A4S auto-scaling system. A4S collects system- and operator-level metrics from the streaming system and uses them to generate memory-parallelism curves. Using these curves, A4S makes scaling and placement decisions that enable meeting the throughput requirements of a streaming job while utilizing resources efficiently and maintaining end-to-end job latency within acceptable bounds.

Section 3.1 describes our assumptions about streaming workloads. Section 3.2 introduces memory-parallelism curves and how they are related to miss-rate curves. Then, Section 3.3 describes our greedy algorithm for choosing operator parallelism and task placement. Finally, Section 3.4 describes our Quickmrc method for generating miss-rate curves efficiently.

3.1 Workload Assumptions

A4S makes two assumptions about streaming workloads:

Low Skew, Data-Parallel Operators: We assume that an operator’s input is hash-partitioned and uniformly distributed across partitions. Each partition is handled by a separate task and the tasks run in parallel. During scaling, when an operator’s parallelism is changed, the repartitioned input still has low skew.

Slowly Changing Workload: We assume that workload characteristics, including the access pattern and throughput, change slowly and remain relatively stable during and after scaling, and so we do not handle short bursty loads. Also, system, including storage, performance remains stable while scaling.

3.2 Memory-Parallelism Curves

Memory-parallelism curves describe the correlation between the memory requirements and parallelism of stateful operators. The intuition for these curves is that for a *given* operator throughput, if we *increase* the parallelism of the operator, the per-task throughput needed *decreases*. As a result, the task can spend more time performing IO and thus requires less cache memory. This argument shows that memory and parallelism can be traded for each other, a relation we express as a memory-parallelism curve for a given throughput.

Section 3.2.1 describes the relationship between memory and parallelism, then Section 3.2.2 shows the benefit of memory-parallelism curves with an example. Finally, Section 3.2.3 explains how we determine a suitable range of parallelism for memory-parallelism curves.

3.2.1 The Relation Between Memory and Parallelism

To understand the relationship between memory and parallelism, we start by considering the impact of memory size on the throughput of operators. Operators process input records, access and update intermediate state, and then output records. The operator state is cached in memory but cache misses cause significant IO latency. The average latency of processing a single record that misses in the cache l_{miss} is mainly due to IO latency. This latency is much higher than (typically 1000 times) the average latency of processing a record that hits in the cache l_{hit} . Let the miss-rate curve $MRC(M)$ be the miss rate of the operator cache at a given memory size M . Then, the average processing latency of a record is:

$$l = MRC(M)l_{miss} + (1 - MRC(M))l_{hit} \quad (3.1)$$

Assume that the operator executes with a parallelism level of P , i.e., with P tasks. Then, based on Little's Law [45] and assuming that the operator does not wait for its input or output, the operator's throughput T is:

$$T = \frac{P}{l} = \frac{P}{MRC(M)l_{miss} + (1 - MRC(M))l_{hit}} \quad (3.2)$$

When making scaling decisions, we aim to ensure that the operator's throughput is greater than or equal to the operator's target throughput T_{target} :

$$\begin{aligned} T_{target} &\leq T \\ &\leq \frac{P}{MRC(M)l_{miss} + (1 - MRC(M))l_{hit}} \end{aligned} \quad (3.3)$$

From Equation 3.3, we can solve for $MRC(M)$:

$$MRC(M) \leq \frac{(P/T_{target}) - l_{hit}}{l_{miss} - l_{hit}} \quad (3.4)$$

Equation 3.4 shows the maximum miss rate of the cache for a given level of parallelism P and target operator throughput T_{target} . With the miss-rate curve $MRC(M)$, we can invert it to find the minimum operator memory M needed to satisfy the miss rate shown in Equation 3.4.¹ We refer to this minimum memory needed to achieve a certain throughput T for a given level of parallelism P as the memory-parallelism curve, or $M = MPC(P, T)$. Each operator has its own set of memory-parallelism curves, based on its miss-rate curve, average request hit/miss latency and throughput.

3.2.2 Understanding Memory-Parallelism Curves

Figure 3.1 shows two memory-parallelism curves for the join operator in the NEXMark Q4 query from our evaluation. Assume that the join operator's throughput is 75000 records/s, its current level of parallelism is 3, and its cache is allocated 1.6 GB of memory (0.53 GB per task). MPC_1 shows the memory-parallelism curve

¹The minimum memory needed is, by definition, a non-increasing function will respect to the miss rate.

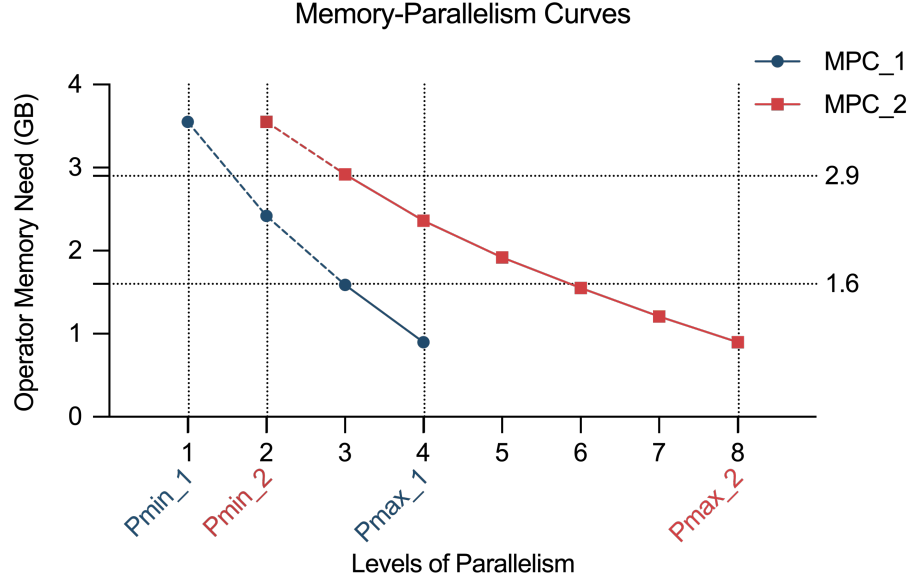


Figure 3.1: An example of memory-parallelism curves. MPC_1 and MPC_2 are two memory-parallelism curves for a join operator with two target throughputs, 75000 and 150000 records/s.

at 75000 records/s, derived using Equation 3.4. This equation shows that for a given operator throughput, as the parallelism P increases, the operator can sustain a higher miss rate. With increasing parallelism, the per-task target throughput decreases and so each task can spend more time on IO, which leads to lower overall operator cache memory requirements.

Equation 3.4 also shows that we can scale the throughput of an operator by scaling its parallelism without changing its cache memory size, i.e., if the ratio P/T_{target} is the same before and after scaling, then $MRC(M)$ and thus M remain the same before and after scaling. Thus, given a memory-parallelism curve at throughput T , we can determine the curve at any target throughput $T_{target} = K * T$, by horizontally scaling the MPC curve by a factor K , or:

$$MPC(K * P, K * T) = MPC(P, T) \quad (3.5)$$

For example, say that the system needs to scale the operator to achieve a target throughput of 150000 records/s. MPC_2 shows the memory-parallelism curve at 150000 records/s, derived by horizontally scaling MPC_1 by a factor of 2, based on Equation 3.5.

In this example, if the cache memory size and the impact of IO on stateful operators is not considered, then one would naively scale the operator by changing its level of parallelism to 6, and simply keep the memory allocation per task unchanged at 0.53 GB, i.e., a total memory allocation of 3.2 GB for the operator. However, this increased overall memory allocation for the operator increases per-task throughput, thus requiring lower parallelism. Based on MPC_2 , 3.2 GB memory allocation suffices for the current level of parallelism, i.e., 3. Alternatively, with a level of parallelism of 6, the total operator memory allocation remains unchanged at 1.6 GB. Ignoring the impact of memory allocation and IO results in either wasteful allocation of 3 tasks or 1.6 GB of memory.

When scaling an operator to a target throughput of 150000 records/s, we can choose any level of paral-

lelism and corresponding memory allocation shown on MPC_2 for the operator. This example shows that memory-parallelism curves provide flexibility with scaling and placement, which we explore in Section 3.3.

As shown in Equation 3.4, memory-parallelism curves are derived using miss-rate curves. In some applications, the miss-rate curve may have performance cliffs where the miss rate changes dramatically with small changes in memory size [47]. In particular, a slight increase in cache size significantly reduces the miss rate. Such cliffs result in relatively flat or “plateau” regions in memory-parallelism curves, where scaling is ineffective because increasing the level of parallelism leads to a small reduction in memory requirements. Section 3.3 describes how the operator scaling and placement decisions in A4S avoid the plateau regions of memory-parallelism curves.

3.2.3 Range of Memory-Parallelism Curves

Memory-parallelism curves provide a range of parallelism levels for achieving a target throughput. With increasing parallelism, per-task memory needs are reduced, which eases placement. However, each task performs more IO, which increases operator latency. We choose a range of parallelism levels $[P_{min}, P_{max}]$ for memory-parallelism curves that ensure that operator latency remains within acceptable bounds. Since operator latency depends on the operator’s throughput, the parallelism range also depends on the operator’s target throughput, as shown in Figure 3.1.

We derive the minimum level of parallelism by assuming that a stateful task has sufficient cache memory so that it can execute without performing any IO. In practice, caches have compulsory misses, but we assume that the operator cache is warm and has sufficient memory to store the operator’s working set. In this case, $MRC(M) = 0$, and so from Equation 3.4, the minimum level of parallelism $P_{min} = l_{hit} * T_{target}$. With this minimum parallelism, the task runs at its maximum throughput and has the lowest latency since it does not perform any IO operations.

As defined, P_{min} can be fractional, but we can only assign an integral number of tasks to an operator. So we define $P_{min} = \lceil l_{hit} * T_{target} \rceil$. At this minimum parallelism level, while cache misses may occur, the operator will still meet its target throughput.

With increasing parallelism, the operator requires less memory but has higher latency. The maximum level of parallelism of an operator should ensure the end-to-end latency target of the job. We assume that the user can determine the maximum operator latency l_{max} based on the job latency. Substituting l_{max} for l in Equation 3.1, the maximum miss rate is $(l_{max} - l_{hit}) / (l_{miss} - l_{hit})$. Then, substituting this maximum miss rate for $MRC(M)$ in Equation 3.4 yields the maximum level of parallelism $P_{max} = \lceil l_{max} * T_{target} \rceil$.

The P_{min} and P_{max} equations show that the parallelism range scales roughly linearly with the target throughput. When scaling an operator, we use a level of parallelism in the $[P_{min}, P_{max}]$ range, as shown in Figure 3.1. In practice, there are two reasons that we may use a minimum level of parallelism that is higher than P_{min} . First, each operator task is placed on a worker, as described later in Section 3.3. A worker may have a maximum memory limit for a task based on the memory available at the worker. If the task’s memory limit is lower than the per-task memory needed to achieve no misses, then we increase P_{min} using Equation 3.4. For example, the per-task memory limit is 1 GB in Figure 3.1. Thus, scaling is performed in the solid curve range, where the per-task memory needs are lower than 1 GB.

Second, as discussed later in Section 3.4, Quickmrc generates partial miss-rate curves up to a maximum operator memory size. We also use this memory limit to adjust P_{min} , thus ensuring that scaling is performed within the known range of the miss-rate curve.

If a task is allocated a minimum amount of memory or the miss-rate curve is inaccurate for low memory sizes, then we can reduce P_{max} as well. However, these limits are not reached in our experiments.

3.3 Co-Designing Scaling and Placement

Memory-parallelism curves provide flexibility with scaling and placement because they allow trading parallelism for memory allocation. As shown in Figure 3.1, an operator’s throughput can be scaled by increasing its parallelism or memory allocation in several ways. For example, it can be scaled by increasing its parallelism and allocating additional tasks when CPU resources are available, or it can be scaled by increasing its cache memory size, which reduces expensive IO operations, when memory is available.

This scaling flexibility poses challenges for task placement. Traditionally, placement is performed using a variant of bin packing, with an integer-linear programming formulation, regardless of the placement objective, including throughput, end-to-end-latency, etc. Each operator’s task is considered an item with a fixed CPU and memory size and the number of such items represents the level of parallelism of the operator. Similarly, a worker is viewed as a container with a fixed CPU and memory size. Then tasks are placed on workers, while optimizing the placement objective. However, with memory parallelism curves, each stateful operator has several memory and parallelism choices, and so the CPU and memory size of items is no longer fixed, significantly expanding the space of choices available for placement.

We simplify and co-design scaling and placement with a greedy algorithm that aims to meet the target throughput of all the operators of a job while minimizing the number of workers or hardware resources allocated to the job.

Algorithm 1 presents the A4S scaling and placement algorithm for one job. It takes the job’s DAG meta-data, and maximum latency and target throughput of each operator of the job, and it returns the parallelism and placement of each operator.

A4S derives the operator’s target throughput from the target throughput of the job using the job’s DAG model (not shown in the algorithm). Similar to DS2, we assign the estimated output rate of an upstream operator as the target throughput (or target input rate) of its downstream operator. For example, suppose that before scaling, an operator’s target throughput is T_{target} , its current input rate (sum of its tasks’ input rate) is R_{input} , and its current output rate (sum of its tasks’ output rate) is R_{output} . Then the target throughput of the downstream operator is $T_{target} * (R_{output}/R_{input})$. Given the target throughput of all the source operators, we calculate the target throughput of the rest of the operators along the edges of DAG.

The A4S algorithm operates in two stages, prepare and execute. In the prepare stage, we estimate the memory and CPU requirements of operators. The `generateMPC` function generates the minimum and maximum parallelism and the memory-parallelism curve for each operator, as discussed in Section 3.2. To estimate the CPU needed by each operator, we assume that it increases linearly with its input rate, regardless of parallelism and the number of IO operations. Thus we estimate it by linearly scaling the operator’s current CPU usage based on its target throughput and current input rate.

In the execution stage, we start by estimating the minimum number of workers W needed to run the job based on the total CPU needs of the operators. This algorithm assumes that all workers have the same number of cores C (and memory size M , as discussed later), which is reasonable because machines with identical configurations simplify management in cloud environments. However, our algorithm can also be extended to handle diverse workers.

We start task placement by initially assigning the minimum parallelism to each operator, which cor-

Algorithm 1: Main Algorithm

```

In : DAG, job's DAG
      N, number of operators
      max_latency[N],
      target_throughput[N]
Out: parallelism and placement of each operator
1 Function A4S:
   // P_min[N] is min parallelism of ops
   // P_max[N] is max parallelism of ops
   // MPC[N] is MPC of operators
2 (P_min[N], P_max[N], MPC[N]) = generateMPC(DAG, N, max_latency,
   target_throughput)
   // CPU_usage[N] is the measured cpu usage of operators
   // input_rate[N] is the measured input rate of operators
   // CPU_needed[N] is the cpu needed by operators after scaling
3 foreach op in [0 ... N) do
4   CPU_needed[op] = CPU_usage[op] * (target_throughput[op] /
   input_rate[op])
5 CPU_total = SUM(CPU_needed)
   // C: nr of cores per worker
   // W: min number of workers needed
6 W =  $\lceil \text{CPU\_total} / C \rceil$ 
7 P = P_min
8 while true do
9   (done, placement) = place(P, MPC, CPU_needed, W)
10  if not done then
11    done = increase_parallelism(P, P_max, MPC)
12    if not done then
13      P = P_min
14      W = W + 1
15  else
16    return (W, P, placement)

```

responds to the maximum memory needed by each operator and least operator latency. Then the `place` function places the tasks of the operators on the W workers. If this placement is successful, the main algorithm returns the number of workers needed and the parallelism and placement of each operator. Otherwise, the tasks cannot fit with their current parallelism and memory needs on the W workers. In this case, the `increase_parallelism` function increases the parallelism of one operator, thus reducing its memory needs, and then placement is rerun. When all operators are at their maximum parallelism, then their parallelism cannot be increased. In this case, we increase W to allocate one more worker, and restart placement with minimum parallelism for all operators.

Our greedy algorithm starts with the minimum number of workers and allocates more workers until the tasks can be placed on them, thus optimizing for the number of workers. By starting with minimum parallelism, we also optimize for operator latency and maximum parallelism provides a latency bound.

The `place` function, shown in Algorithm 2, places tasks on workers based on each operator's memory and CPU needs and returns its placement. The placement indicates, for each task, the worker on which to place the task, and the CPU and memory needed by the task. The function begins by initializing the `available` array that records the percent CPU (1 core = 100% CPU) and memory available on each worker. Then, we calculate the CPU and memory needed for each task. For a task's CPU needs, we divide its operator's CPU needs by its current operator parallelism. For a task's memory needs, we use the

Algorithm 2: Place Algorithm

```

In : N, number of operators
      P[N], current parallelism of operators
      MPC[N], MPC of operators
      CPU_needed[N], cpu needed by operators
      W, current number of workers
Out: placement of tasks
1 Function place:
   // C: cores available per worker
   // M: memory available per worker
2   available[W] = [(C, M), (C, M), ...]
   // number of tasks of all operators
3   T = SUM(P)
   // (cpu, mem) needed by all T tasks
4   needs[T] = []
5   foreach op in [0 ... N) do
6     p = P[op]
7     cpu = [ CPU_needed[op] / p ]
8     mem = [ MPC[op][p] / p ]
9     foreach op_task in [0 ... p) do
10    | needs.push((op, op_task, cpu, mem))
11  Sort needs in decreasing order of memory needed
12  placement = []
13  foreach task in needs do
14    find worker w with largest available[w].mem where
15    task.cpu < available[w].cpu and
16    task.mem < available[w].mem
17    if w does not exist then
18    | return false
19    placement.push(task, w)
20    available[w].cpu -= task.cpu
21    available[w].mem -= task.mem
22  return (true, placement)

```

memory-parallelism curve of the operator to determine the operator's memory needs for the current level of parallelism and then divide it by this parallelism. We allocate the same CPU and memory to the tasks of an operator because we assume low skew across these tasks.

The tasks of all the operators and their CPU and memory needs are stored in the *needs* array, which is used during placement. It is sorted by the task memory needed in decreasing order to prioritize placement of tasks with larger memory needs. Each task is placed on a worker that has the most amount of memory available and enough CPU and memory for the task. This worst-fit scheme balances memory usage across workers. The *place* function succeeds when all tasks can be placed on the current number of workers.

While not shown, the A4S algorithm handles stateless operators as well. These operators have fixed parallelism and limited memory needs. In the A4S Algorithm 1, we add the CPU needs of stateless operators to CPU_{total} . In the *place* function, after placing the stateful tasks, we place stateless tasks on workers with the most CPUs available (*cpu.left*).

The *increase-parallelism* function, shown in Algorithm 3 increments the parallelism of one operator to reduce its memory needs. The function increases parallelism by one, until the maximum parallelism of the operator. It greedily chooses an operator whose memory needs are reduced the most when its parallelism is increased. This greedy choice avoids choosing parallelism levels in the plateau regions of the memory-parallelism curves (see Section 3.2.2), where memory requirements do not decrease significantly.

Algorithm 3: Increase Parallelism Algorithm

```

In : N, number of operator
      P[N], current parallelism of operators
      P_max[N], max parallelism of operators
      MPC[N], MPC of operators
Out: updated parallelism P[]
1 Function increase_parallelism:
    // Find an operator whose memory needs decrease most when its
    // parallelism is increased by 1
2   reduced = 0, index = 0
3   foreach op in [0 ... N) do
4       mpc = MPC[op]
5       if P[op] < P_max[op] then
6           diff = mpc[P[op]] - mpc[P[op] + 1]
7           if diff > reduced then
8               reduced = diff, index = op
9   if reduced == 0 then return false
10  P[index] = P[index] + 1
11  return true

```

Scaling Limitations Our scaling algorithm has two limitations. First, the placement algorithm currently does not account for task and operator locality and so during placement, a task operating on a worker may be moved to another worker and the tasks of an operator may be placed on any worker. We plan to explore these locality optimizations in the future.

Second, A4S requires runtime operator metrics to make scaling decisions. When a job starts up, its metrics are not available, and so we assume that a static configuration is used for scaling and placement. For example, an operator can be assigned a parallelism level of one and randomly placed on any worker that has some minimum memory available.

3.4 Quickmrc

This section describes Quickmrc, an online method for generating miss-rate curves. We use these curves within the `generateMPC` function in the A4S Algorithm 1 to determine memory-parallelism curves based on Equation 3.4.

The design of Quickmrc is driven by our goal of generating miss-rate curves with minimal overhead and reasonable accuracy. When making scaling decisions, we prefer to use the miss-rate curve of the current workload, so we need to generate the miss-rate curve for a recent period, e.g., last 10 minutes. Traditionally, stack distance calculation has been expensive, and so many existing solutions stream access traces to a separate storage service and then simulate cache operations to perform stack distance analysis on the traces. For example, MIMIR [58] and fixed-rate sampling in SHARDS [66], which we leverage in Quickmrc, use a separate storage service and generate offline miss-rate curves. While this approach offloads stack distance calculation, it requires significant communication for sending the access traces, additional costs for storing and analyzing the traces, and sacrifices stack distance freshness. Instead, we use Quickmrc to generate and pre-process stack distance information at each operator task. Then, Quickmrc merges this information to generate the operator miss-rate curve.

With online stack distance generation, we trade some accuracy for low overhead. Quickmrc builds on two efficient approximation methods, grouping accesses [58] and sampling accesses [66]. We overestimate stack

distances and thus memory requirements, which is acceptable in streaming environments because it helps tolerate bursty streams.

Section 2.2 provides background on stack distance and existing miss-rate generation methods. Section 3.4.1 describes how Quickmrc generates stack distance efficiently and then Section 3.4.2 explains how we use stack distance information to generate the miss-rate curve.

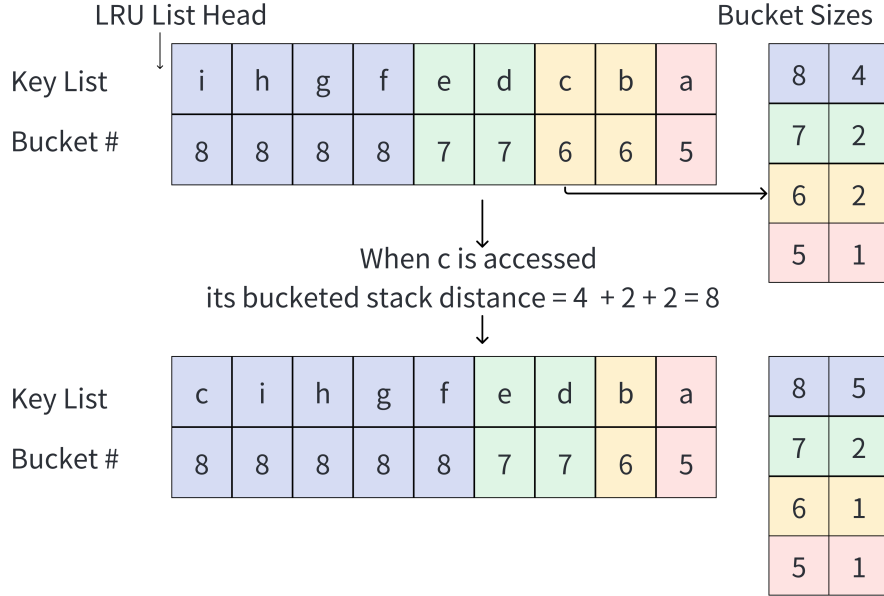


Figure 3.2: Estimating stack distance based on bucket sizes.

3.4.1 Generating Stack Distance

Quickmrc assumes an LRU cache. Recall from Section 2.2, when an item in an LRU cache is accessed, the stack distance of the item is the depth of the item from the top of the LRU stack. In streaming processing workloads, a new record (a record with a key that has never been seen previously) will cause a cache miss and lead to accessing storage. However, these accesses do not cause IO operations because the storage layer filters such accesses using bloom filters. Thus, we ignore the stack distance of such accesses.

Given a set of accesses, their stack distances enable calculating the miss-rate curve, which is the cache miss rate at different memory sizes. Next, we describe stack distance calculation in Quickmrc.

Bucket-based LRU A standard, accurate miss-rate curve implementation maintains a balanced tree of unique items and tracks the most recent reference to each item [54]. The tree is ordered by the last reference time, and each item stores the total number of items in the subtree rooted at the time. When an item is accessed, a separate hash table is used to lookup the item in the tree, and then the stack distance is computed by traversing the tree from the item to the root of the tree in $O(\log(M))$ time, where M is the number of (unique) items in the tree. Then, the item is removed from the tree and re-added with the latest reference time, which may require rebalancing the tree.

We reduce these costs of determining the stack distance on each access by bucketing the cache entries in the LRU cache, similar to MIMIR [59]. We logically divide the LRU cache into variable-sized cache buckets,

each containing consecutive cache items, as shown in Figure 3.2. Key i at the head of the LRU list is most recently accessed, and Key a at the tail of the LRU list is least recently accessed. The figure shows 4 buckets, numbered 5-8. Each key is assigned a bucket, and a bucket array maintains bucket sizes, i.e., the number of keys in each bucket. For example, the size of Bucket 8 is 4. When Key c is next accessed, its stack distance is 7. We approximate its stack distance by summing the bucket sizes of the buckets starting from the list head (most recent bucket) until the bucket associated with the key. Thus, the bucketed stack distance of Key c is $4 + 2 + 2 = 8$.

We update a pair of buckets on each cache access (assuming that the key exists in the cache). For example, as shown in Figure 3.2, when Key c is accessed, we move it from Bucket 6 to Bucket 8 (the most recent bucket) by updating the bucket number associated with the key. We also update the size of both these buckets. When an item is evicted from the cache, we decrement the count of its corresponding bucket.

We create a new bucket when the most recent bucket reaches a maximum size limit. The new bucket becomes the most recent bucket. Over time, the most recent bucket grows while the rest of the buckets shrink in size, either due to evictions or items being moved to more recent buckets due to accesses.

The complexity of the stack distance calculation is $O(B)$ with B buckets. In practice, the number of buckets is relatively small. For example, in our implementation, the default value of B is 60. Thus, the stack distance calculation can be performed efficiently with a fast, linear pass over the bucket array. In principle, the bucketed stack distance calculation can be performed in $O(\log(B))$ time by using a balanced tree implementation, similar to regular stack distance calculation. However, we do not expect it to perform any better since B is relatively small.

The bucketed stack distance overestimates the stack distance, and its inaccuracy is bounded by the maximum bucket size limit. Thus, assuming B buckets, the worst-case error is roughly $1/B$. For example, the worst-case error in our implementation is $1/60$ or 1.6%. Our experiments show that the average error is significantly lower because the buckets are variable-sized, and many buckets have few items.

Quickmrc makes two modifications to MIMIR’s bucketing strategy. First, while MIMIR interpolates the stack distance within a bucket, we simply use the bucketed stack distance, which is more efficient. Second, MIMIR uses a fixed number of buckets, which requires aging (reusing) buckets. Their ROUNDER aging policy merges the least recent two buckets and makes the least recent bucket available for a new bucket. However, each successive merge increases the count of the least recent bucket, leading to increasing inaccuracy for large stack distances.

To provide higher accuracy when scaling out, i.e., for larger memory sizes, Quickmrc uses a variable number of buckets. With increasing buckets, the stack distance calculation time increases as well. To avoid unlimited buckets, we keep track of the current set of buckets, and when a bucket and *all* earlier (less recent) buckets become empty (i.e., they have a zero count), we reuse them. While this is sufficient for our experiments, we can further limit the number of buckets by periodically garbage collecting other empty buckets. A more sophisticated policy would choose a pair of consecutive buckets with the smallest sum of bucket sizes and merge them into a single bucket (and garbage collect the other bucket), which would balance the load across buckets over time.

With this design, we limit the stack distance calculation time to $O(B)$, where B is the number of buckets. This complexity is not necessarily lower than the $O(\log(M))$ complexity of tree-based schemes such as Search trees [55] and Interval trees [10]. However, the bucket counters are stored consecutively in memory and thus have better data locality and lower access latency. In addition, the update complexity of the bucketing scheme is $O(1)$, which is much better than $O(\log(M))$ for the tree-based schemes.

Ghost Cache Until now, we can only calculate the stack distance of the items in the LRU cache, i.e., whose stack distance is within the current cache size. If an item that was previously evicted is accessed again, we need to treat it as a newly accessed item. Since we cannot estimate its stack distance, we cannot determine the miss rate of the cache beyond the current cache size, but we need it for scaling out. We mitigate this issue by keeping the evicted keys in a ghost LRU cache, which enables tracking the stack distance of the evicted items.

When an item in the LRU cache is evicted, its value is deleted, but its key is inserted in the ghost cache. When a key in the ghost cache is accessed, the cache access fails, but the key’s bucketed stack distance is calculated similarly to the items in the LRU cache, as described above. The overhead of managing the ghost cache is small. It is accessed when items are evicted from the LRU cache or when the keys in the ghost cache are accessed. Calculating the stack distance for when the ghost cache keys are accessed incurs slight performance overhead, but it is negligible compared with storage latency on a cache miss.

We limit the ghost cache size by storing a fixed number of evicted keys, which is a multiple G of the number of the items in the LRU cache. Assume that the number of items in the LRU cache is L . Then, the number of keys in the ghost cache is $G * L$, and the maximum stack distance D_{max} that can be calculated is $L * (1 + G)$.

The choice of G represents a tradeoff between the range of scaling options and cache space overhead in A4S. With increasing G , the miss-rate curve can be generated for a larger $(1 + G)$ times memory size until the miss rate reaches 0. Recall from Section 3.2.3 that we may increase the minimum level of parallelism P_{min} based on the memory size limit of the miss-rate curve generated by Quickmrc. Thus, with increasing G , we have a larger range of parallelism levels for scaling. However, increasing G also increases the cache space overhead. Assume that the average key size is KS and the value size is VS . Then, the ghost cache space overhead is $G * KS / (KS + VS)$. This overhead is low when the value size is much larger than the key size in streaming workloads. For example, the value size is more than $100\times$ the key size in the NEXMark Q4 query. In our experiments, we find that a value of G between 1 and 2 is sufficient for scaling throughput by a factor of roughly 10, and so the cache space overhead is roughly 1-2%.

Fixed-Rate Spatial Sampling As mentioned above, the complexity of calculating the bucketed stack distance on each access is $O(B)$. We reduce this cost further by employing the fixed-rate spatial sampling approach, described in SHARDS [66], which samples the stack distance for a randomly selected set of items. Our sampling is straightforward and selects 1 in K items. When a key is accessed, we compute $h = \text{hash}(\text{key}) \bmod K$, select the key when $h \leq 1$, and then compute the stack distance for a selected key. Whether a key is selected or not, we update its bucket and the corresponding bucket sizes. The SHARDS work showed that sampling 0.1-1% keys is sufficient for good accuracy. We sample between 1-3% keys in our experiments.

Our fixed-rate sampling approach has subtle differences from SHARDS because we generate partial miss-rate curves, while SHARDS generates full miss-rate curves. As a result, with fixed-rate sampling, SHARDS needs to store all the sampled keys, which grow over time, leading to similar complexity as non-sampling schemes. In contrast, Quickmrc’s LRU and ghost caches have fixed sizes. SHARDS also supports fixed-size sampling, but this requires adapting the sampling rate and evicting samples using a priority queue, both of which add 3-4x overhead over fixed-rate sampling. Quickmrc relies on LRU eviction already used by the operator caches.

We use fixed-rate sampling for the LRU cache. For the ghost cache, currently, we do not use sampling for

d (or n)	$h(d)$	$HR(n)$	$MR(n)$
1	5	5/10	5/10
2	3	8/10	2/10
3	2	10/10	0

Table 3.1: Calculating the unscaled miss rate curve using merged stack distances. $h(d)$ is the number of cache hits with stack distance d . $HR(n)$ and $MR(n)$ are hit rate and miss rate with cache size n .

two reasons. First, as mentioned previously, stack distance calculation time for the ghost cache keys is small compared to storage latency. Second, it allows deriving a more accurate miss rate curve for the ghost cache memory range, which is particularly useful when scaling out. Since we sample 1 in K keys from the LRU cache but do not sample the ghost cache, we scale the counts of the sampled items by K since each sampled item represents a larger number of items. Unlike SHARDS [66], we do not scale the stack distance of the sampled items because the LRU cache stores all the items.

In the future, we plan to explore sampling the ghost cache. The benefit of this approach is that we can avoid storing the unsampled keys in the ghost cache. The reduced ghost cache space overhead will be especially beneficial when the average value size is closer to the key size.

3.4.2 Generating Miss-Rate Curve

A4S uses Quickmrc to efficiently generate the stack distance of an item when it is accessed, as described in Section 3.4.1. This section describes how we use this stack distance information to generate the operator’s miss-rate curve.

An operator is assigned a set of tasks. Each task operates on a partition of the keys of the operator’s input stream and calculates stack distance information independently. To merge this information, we need to understand the relationship between the task stack distance and operator stack distance. The stack distance sd of a key k represents the number of unique items between two consecutive accesses to this key. Assume that the keys are hash partitioned for load balancing and thus uniformly distributed across P partitions (or number of tasks). The stack distance of key k within the partition that contains this key will be sd/P because only $1/P$ items of the operator are assigned to this partition. Thus, the operator stack distance can be estimated by multiplying the task stack distance with the number of partitions P . Alternatively, the miss-rate curve of the operator $MRC(M)$ can be derived by horizontally scaling the miss-rate curve of a task $MRC_{task}(m)$ by a factor P , i.e., $MRC(m * P) = MRC_{task}(m)$, where m is the task’s cache memory size.

Figure 3.3 shows an example of how A4S calculates the miss rate of an operator that has two tasks. Each task generates its own stack distance stream. The calculation proceeds in three steps. First, we merge the per-task streams into a single stack distance stream. Then, we use this stream to generate an unscaled miss-rate curve. Finally, based on the analysis above, we scale this curve by the current number of tasks (2 in this example) to generate the operator’s miss-rate curve.

The miss rate shown in Figure 3.3 is based on the number of keys. We scale the miss rate for memory size by multiplying the stack distance with the average item size, i.e., the sum of the key and value sizes. Although the item size varies, it is small compared to the cache size, and so any inaccuracy in the miss-rate curve due to this approximation is small.

The cache hit rate $HR(n)$ for a cache of size n is calculated after a set of accesses (say M) by tracking the number of accesses with stack distance at most n . In particular, $HR(n) = \sum_{d=0}^n h(d)/M$, where $h(d)$ is

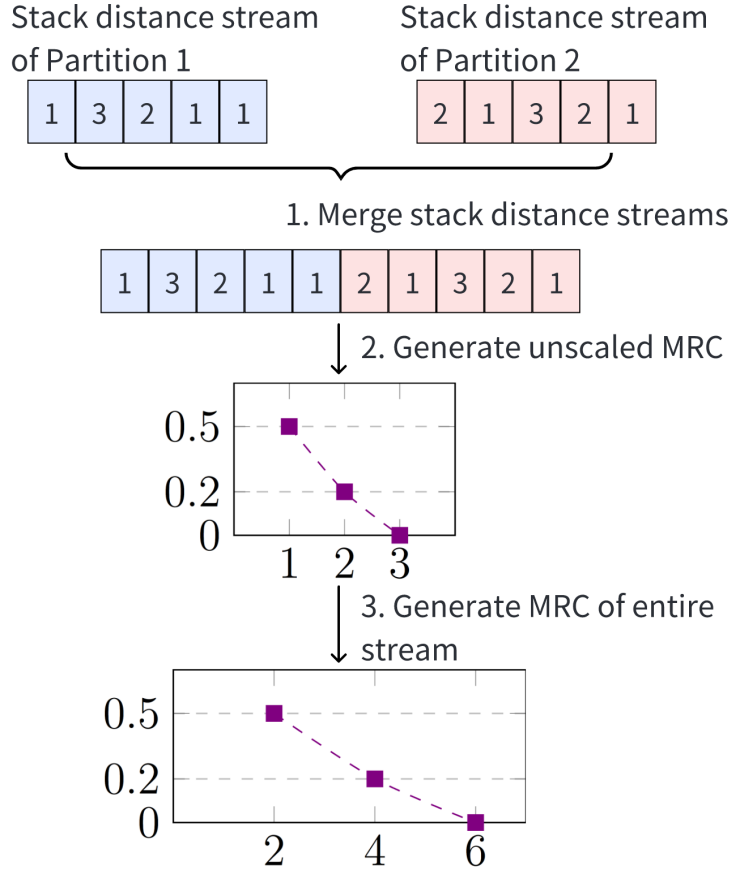


Figure 3.3: Estimating the miss rate curve of an operator from the stack distance streams of its two tasks.

the stack distance frequency distribution, i.e., number of hits at stack distance d and n ranges from $0 \dots M$. Then, the miss-rate curve is determined using $MR(n) = 1 - HR(n)$.

Table 3.1 shows how we use the merged stack distance stream to calculate the unscaled miss-rate curve (Step 2 in Figure 3.3). Column $h(d)$ shows the stack distance frequency distribution. Column $HR(n)$ shows the hit-rate curve, and Column $MR(n)$ shows the miss-rate curve. The per-task stack distance streams can be merged in any order in Step 1 in Figure 3.3 since we only use the stack distance frequency distribution.

We have described how the stack distance streams of an operator's tasks are used to calculate the operator miss-rate curve. We reduce communication by having each task generate a stack distance histogram. Unlike LRU buckets, whose sizes vary over time, we split the stack distance range $[0, D_{max}]$, where D_{max} is the total number of keys in the LRU and ghost caches, into H fixed-size histogram buckets and record stack distance frequency per bucket. Then, we merge and scale the per-task histograms to determine the operator miss-rate curve. With a histogram, the maximum error in the stack distance calculation is $1/H$. In our implementation, the default value of H is 50, so this error is 2%.

Chapter 4

Implementation and Deployment

We have implemented A4S for the cloud-native Risingwave stream processing system (v0.18.0). Figure 4.1 shows the Risingwave architecture. Risingwave consists of four components: a frontend, compute nodes, compactors, and a Meta server. The frontend parses SQL queries from clients and performs planning and optimizations of query jobs. The compute nodes are the Risingwave workers that perform data computation. They cache intermediate state and spill this state as LSM-type files to a persistent store, such as AWS S3. The compactors read these files from the store and merge them to create new, compacted files. Compaction gets rid of old data versions and improves read performance. The Meta server coordinates operations between the frontend and compute nodes and assigns compaction tasks to compactors. The blue arrows in the figure show data communication with and between the components of Risingwave. Typically, a separate metrics server, such as Prometheus, is deployed to monitor the status of the cluster. The metrics server is a time series database that timestamps inputs and supports aggregation operations over a time interval. The system is deployed on Kubernetes.

Each compute node uses two types of caches, an operator cache and a block cache. When an operator accesses data, it first looks up the operator cache and then the block cache. If both miss, the operator fetches data from the persistent store. Operator caches are designed based on the operator semantics. For example, the cache key for a join operator cache is the join key, and the cache value is the set of records of associated with the join key. The block cache caches the blocks of the files on the persistent store and it is shared among all the operators on the compute node. The block cache key is a storage block key (file id + block id in file). The block cache value is a sequence of sorted key-value pairs. In Risingwave, the key in each key-value pair is prefixed by a partition id to support recovery after repartitioning. However, this design leads to poor locality in the block cache (e.g., two nearby keys may be located in different blocks when they have different prefixes) so the block cache miss rate is generally high (e.g., 90% in the join operator of NEXMark Q4). Hence, A4S's memory scaling focuses on the operator cache. We allocate a fixed amount of memory for the block cache and consider the miss rate of the block cache as constant in our experiments.

We implement A4S as an independent controller component in Risingwave, which we expect will simplify deploying it for other streaming systems. The A4S component interacts with Risingwave and Prometheus to determine the parallelism and placement for operators on compute nodes so that the operators satisfy their target throughput and latency requirements, using the fewest number of compute nodes. The orange arrows in Figure 4.1 show the inputs to A4S and the red arrow shows the output of A4S, which the Meta server uses to perform scaling and placement. A4S assumes that the persistence store provides unlimited IO bandwidth

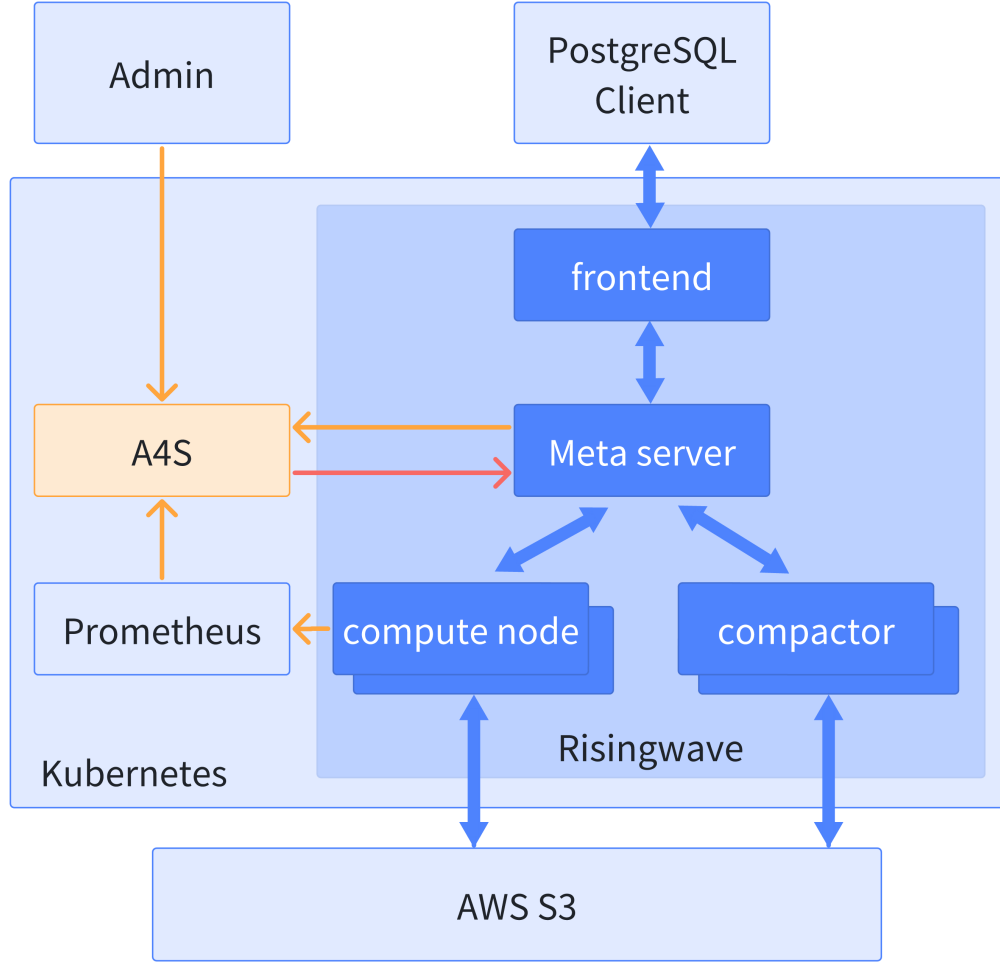


Figure 4.1: The Risingwave Architecture with the A4S Controller.

and storage and stable IO latency, thus does not optimize for them.

A4S needs the stack distance histogram, CPU usage, cache hit and miss latencies, cache item size, and input and output rate of each operator. We modify the LRU cache implementation in the compute nodes to support stack distance calculation, as described in Section 3.4.1, and collect per-task stack distance histograms (by default, 50 bucket values), as described in Section 3.4.2. To estimate CPU usage and cache latencies, we measure the number of IOs per second R_{io} , the IO time per second T_{io} , and the wait time T_{wait} per second for each task. The wait time is the time a task waits for its upstream input and downstream output. Then, we estimate CPU usage as $1 - T_{io} - T_{wait}$ since Risingwave does not provide accurate CPU usage. This estimate may be slightly higher than real CPU usage if a compute node runs background or daemon tasks. Assuming that the current task input rate is R_{input} , we estimate the cache hit latency to be $(1 - T_{io} - T_{wait})/R_{input}$, and the cache miss latency to be the sum of the hit latency and T_{io}/R_{io} . The rest of the metrics are already collected at the compute nodes by Risingwave. Compute nodes report these metrics to the Prometheus server periodically (by default, once per second). This implementation adds 2,000 lines of Rust code at the compute node.

Risingwave’s original operator cache implementation did not support specifying the operator cache size. Instead, Risingwave sets the operator cache size dynamically with an epoch-based eviction policy. When a

compute node is under memory pressure, e.g., when adding an operator cache for a stateful operator, it evicts cached items with a timestamp earlier than an epoch timestamp for all of its operator caches. We modified the operator cache implementation to support size-based eviction as well. A task can specify a maximum operator cache size, in terms of the number of keys, and we use this maximum size to limit the cache size when placing stateful tasks on compute nodes.

The A4S algorithm is currently triggered manually, with a user informing A4S of the target throughput of the source operators and maximum latency of the stateful operators. Ideally, the target throughput would be estimated based on the throughput and the amount of data accumulated in the upstream system, and the maximum operator latency would be determined based on the workload SLO and measured operator latencies. Then, A4S could be triggered automatically at regular intervals to perform scaling and placement.

When A4S is triggered, it retrieves the job’s DAG information from the Meta server. It also retrieves metrics within a recent time interval (by default, 10 minutes) for each task from Prometheus. The stack distance histogram is cumulative and the rest of the metrics are averages over the per-second values reported by compute nodes. We merge these per-task metrics to derive the corresponding operator metrics. We merge and scale the stack distance histograms, as described in Section 3.4.2.¹ We average latencies and cache item sizes, which is reasonable under our low skew assumption, and sum the input and output rates and CPU usage of the tasks.

A4S uses this information to generate memory-parallelism curves for each operator and then outputs the number of compute nodes, the parallelism of each operator, and for each operator task, its memory needs and placement on a compute node, as shown in Algorithm 1. A4S uses this information to invoke the Meta server’s Reschedule API, which changes the parallelism of operators and places the tasks. The Reschedule API allows dynamic reconfiguration of the parallelism and placement of the operators. In particular, after scaling, if a task that is placed at a compute node already exists at that node, then the task is reused, without restarting it.

Risingwave optimizes communication between upstream and downstream operators by using two types of channels between their tasks. It uses a local channel between tasks that are located at the same compute node, and a remote channel between tasks located at different compute nodes. However, based on our evaluation, a downstream operator usually reads data at a faster rate from an upstream operator connected via a local channel versus a remote channel. This difference in rates can lead to misalignment between streams (e.g., join streams), which causes poor locality and performance degradation [32]. As a result, we modify compute nodes to only use remote channels.

The A4S design uses an operator’s miss-rate curve to generate its memory-parallelism curves. Some operators may use multiple operator caches. For example, the join operator has two input streams whose data is cached in two separate operator caches. For each incoming tuple, from either stream, the operator looks up both caches, and so the caches have similar size in the number of keys (and Risingwave’s eviction policy evicts the same keys from both the caches). Other operators may have caches with different sizes. For example, the aggregation operator in NEXMark Q16 uses two distinct operations and thus has three caches, one keyed by the group-by key, and other two keyed by the distinct keys. The sizes of the latter two caches depend on the number of unique keys. Our implementation generates a separate histogram for each cache of an operator. When generating memory-parallelism curves, it estimates an operator’s memory requirements based on linearly scaling the current cache size (i.e., number of items in the cache $\times$ cache item size) of all

¹In some cases, the histogram bucket sizes can be different across an operator’s tasks. The implementation supports merging these histograms as well.

the caches. This approach enables scaling the different caches so that the ratio of their cache sizes remain the same after scaling. However, the current A4S implementation only supports multiple operator caches for the join operator.

Resingwave chains multiple operators into a fragment to reduce the overhead of data communication between operators, and the Reschedule API is fragment-based. A4S currently does not support multiple stateful operators in a fragment, and so we modify the frontend so only one stateful operator is placed in a fragment.

Chapter 5

Evaluation

This chapter evaluates the A4S scaling and placement strategy. Section 5.1 describes our experimental methodology. Section 5.2 compares the A4S and the DS2 scaling and placement methods, and Section 5.3 analyzes the overhead introduced by A4S. Finally, Section 5.4 evaluates the accuracy and overhead of Quickmrc.

5.1 Methodology

5.1.1 Experimental Environment

We use AWS EKS to deploy a Risingwave Kubernetes cluster on a single AWS EC2 c5.12xlarge instance (96 GB of memory and 48 vCores). Each Risingwave component is deployed as a container in a Kubernetes pod, which is the smallest deployable unit in Kubernetes. We use one large EC2 instance instead of several smaller instances to ensure that all the communication between compute nodes occurs within a single machine so that network bandwidth will not become a potential bottleneck.

We choose 4 GB of memory and 2 vCores as the resource limit for a compute node pod, based on the minimum requirement for running streaming queries on Risingwave compute nodes. We choose this minimum size granularity for compute nodes to minimize the over-allocation of memory and CPU resources. A compute node reserves roughly 850 MB for the operator caches, and the rest of the memory is used for the block cache, network buffers, write buffers, channels, and shared libraries. We set the resource limit for the rest of the components to 1 GB of memory and 1 vCore. We deploy 6 compactor pods, 1 frontend pod, and 1 Meta server pod in the cluster, which is sufficient for our experiments. When scaling, additional compute nodes may be needed. To avoid cold starting pods, with unpredictable times, we pre-create all compute node pods that are needed after scaling.

5.1.2 Workloads

For evaluation, we use NEXMark [64], a widely used benchmark suite [36, 44] for queries over continuous data streams. The benchmark contains several queries over a three-entity model representing an online auction system: auction, person, and bid data streams.

A4S is primarily beneficial for scaling queries that use memory-intensive stateful operators. For our evaluation, we choose three typical stateful queries (Q4, Q18, Q20) among the 23 queries in Flink’s extended

Query #	Number of Operators	Stateful Operators	Bottleneck Operator
Q4	6	Join, Aggregation	Join
Q18	2	TopN	TopN
Q20	4	Join	Join

Table 5.1: Queries and their characteristics.

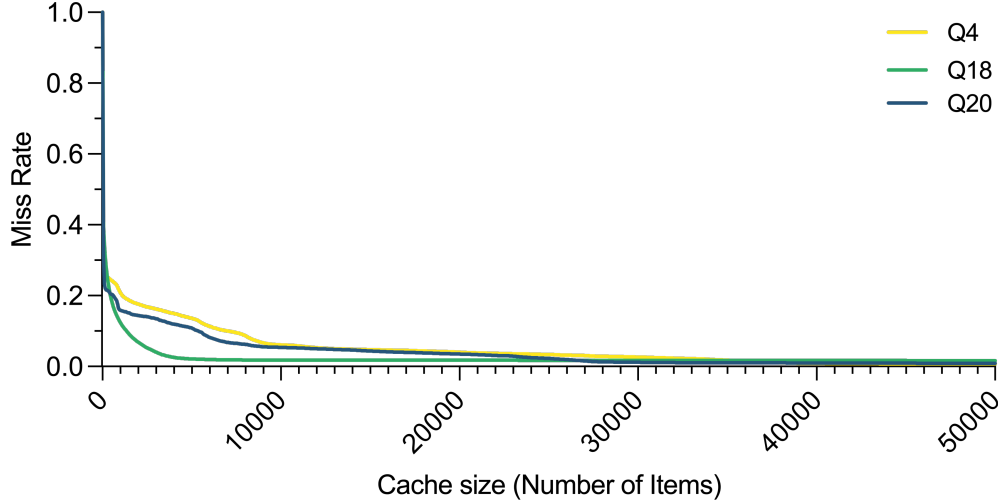


Figure 5.1: Miss-rate curves for the Q4, Q18 and Q20 NEXMark queries.

NEXMark benchmark suite [6]. Q4 calculates the average bidding price for any auction category, Q18 identifies the last bid of a bidder for an auction, and Q20 finds the bids with the corresponding auction information where the category is 10. Table 5.1 shows the characteristics of these queries. Q4 contains 6 operators, three of which are stateful (one join and two aggregation operators), and the bottleneck is the join operator. Q18 contains 2 operators, and the bottleneck is the stateful TopN operator. Q20 contains 4 operators, one of which is stateful, and its bottleneck is the join operator. The bottleneck operator limits the throughput of the entire job and requires the most amount of memory. For example, for Q4 join, the records with the auction join key that are recently accessed are cached and the rest of the join key accesses require IO operations. Similarly, for Q18 TopN, the records with the (bidder, auction) group key that are recently accessed are cached.

The original NEXMark event generator was written in Java and writes events to XML files. It does not support the newer queries in extended NEXMark or storing data to streaming storage systems, such as Kafka, that are typically used as the data source of streaming processing systems. So, we implement a new NEXMark generator in Rust that produces extended NEXMark events in JSON format and sends these events to a Kafka cluster. Then, Risingwave consumes data from Kafka and processes it.

NEXMark Modification In the default NEXMark benchmark suite, the data streams exhibit good locality because record processing only reads intermediate states produced by processing recent records, and so stateful operators rarely access the persistent layer. In practice, while most data has good locality in streaming workloads, some data is delayed [11, 43, 62] because it is routed through networks [42, 62], stored in intermediate storage systems such as Kafka, or due to transaction aborts that occur in upstream systems. As a

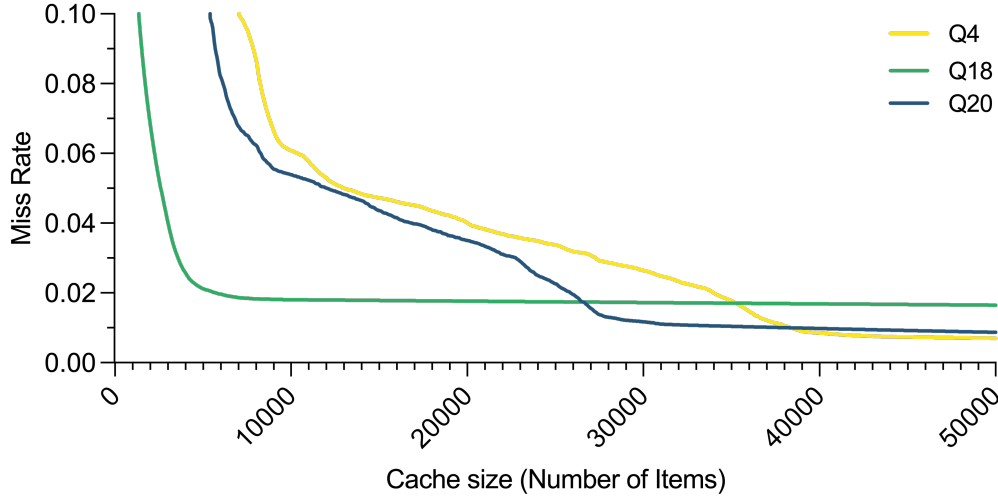


Figure 5.2: Zoomed in miss-rate curves for the Q4, Q18 and Q20 NEXMark queries.

result, streaming workloads consist of a majority of data with good locality and a small amount of data with poor locality that disproportionately affects performance. For example, Figure 5.1 shows the MRC curves for the Q4, Q18 and Q20 workloads. It shows that the miss rate becomes low ($< 6\%$) beyond a small cache size (roughly 10000) and then drops slowly. This behavior shows that there are a small number of events (these events are usually the delayed events) that access old records while the rest of the events access recent records. To handle delayed events, streaming systems such as Millwheel [8], Spark Streaming [73], Google Dataflow [9], and Flink [24] use watermarks to drop records delayed beyond a recent time window.

The default NEXMark setup does not model delays in real-world streaming workloads and so we modified the NEXMark generator to insert delayed records. In our setup, the generator has a 2% chance of randomly generating delayed records. For the queries in our workload, the operator cache can cache data for a 20-second interval. In NEXMark queries, the time window for all the window operators is 2 minutes, and the per-stream watermarks are usually set to several seconds beyond the time window. Thus, we delay records randomly up to a maximum of 2 minutes since any records delayed further will likely be dropped. With this delay modification, the workload has at most 2% misses in the operator cache. These misses decrease a single task's throughput from roughly 100000 records/s to about 19000 records/s for NEXMark Q20 and Q4 and from 85000 records/s to about 6000 records/s for NEXMark Q18, which shows that IO has a significant impact on throughput and caching is critical for good performance. Figure 5.2 zooms into the MRC curves for the Q4, Q18 and Q20 workloads. The miss rate decreases slowly for larger cache sizes but even small decreases in miss rate lead to significant increase in throughput. For example, for NEXMark Q18, the throughput increases by roughly 10000 records/s with every 0.1% decrease in miss rate.

Baseline We compare A4S with DS2, a state-of-the-art scaling solution. DS2 utilizes the DAG model of streaming jobs to calculate each operator's target throughput based on the target throughput of source operators, which we use in A4S as well. Then, DS2 uses the ratio of the target throughput and true rate (input rate assuming no waiting time for input and output) to decide the scaled parallelism of the operator. However, the scaled parallelism does not consider the operator's memory needs after scaling, so it overestimates the operator's memory requirements.

DS2 only provides scaling policies that decide the parallelism of operators. The DS2 work is used for scaling in Flink, so we combine DS2 with Flink’s task placement and use it as our baseline. We implement this baseline in Risingwave to provide a fair comparison with A4S.

In Flink, each worker has multiple slots, one per CPU core. Each slot is allocated a fixed amount of memory, and slots provide memory isolation. Flink uses the maximum parallelism (M) among all operators of a job to assign M slots to the job. Each slot groups at most 1 task from all operators. This ensures that the tasks of each operator can run in parallel. Grouping the tasks of different operators within a slot enables efficient resource sharing between CPU and IO-bound tasks. However, this grouping does not take the CPU usage of tasks into account and so may waste CPU with many IO-bound tasks or degrade performance with many CPU-bound tasks.

Risingwave does not support memory isolation within workers, so our Flink placement implementation does not provide memory isolation between slots. However, in practice, the tasks of the bottleneck operators are placed in different slots, and so they compete with each other. As a result, all the slots use roughly similar memory, providing comparable performance to Flink’s fixed memory slots. In the following text, we will simply refer to the combination of DS2 scaling and Flink placement as DS2.

DS2 outputs a fixed level of parallelism for each operator, and Flink placement requires such a fixed level of parallelism. A4S scaling outputs a range of levels of parallelism, and A4S placement exploits this range for placement. As a result, we cannot mix and match scaling and placement methods.

5.2 A4S Scaling and Placement Performance

5.2.1 Setup

Before running our experiments, we first generate all the source data in Kafka. We use 4 Kafka partitions, which ensures that the Kafka cluster does not become a bottleneck. Then, we start running a query on Risingwave on one compute node and set the initial parallelism of all its operators to one. However, as per Risingwave recommendations, we set the parallelism level for the source operators to be 4, the same as the number of Kafka partitions, and keep it unchanged during scaling. DS2’s Flink placement requires an additional compute node for an operator with 4 tasks (since each compute node has 2 vCores), which is wasteful. So, during initialization, we bypass this placement and place all operator tasks on one compute node. Source tasks may be moved to other compute nodes after scaling.

Risingwave usually needs several minutes, depending on queries and workload, to warm up and provide stable performance. Furthermore, both DS2 and A4S need stable metrics to make their decisions. We trigger scaling in DS2 or A4S roughly 10 minutes after the query throughput becomes stable. For scaling decisions, we use metrics from the past 10 minutes. With the initial level of parallelism of one, we observe that the throughput of Q4 and Q20 is roughly 19000 records/s, so we set the target throughput to 150000 records/s, which is roughly $8\times$ the current throughput. For Q18, the observed throughput is about 6000 records/s, so we set the target to 48000 records/s. We set the maximum operator latency after scaling to be the same as the operator latency before scaling.

5.2.2 Results

Table 5.2 shows the A4S and DS2 scaling decisions for different queries. The parallelism column shows the parallelism of the bottleneck operator in the query before and after scaling. The parallelism of all other

Query#	Scaler	Parallelism	Worker Num
Q4	A4S	$1 \Rightarrow 6$	$1 \Rightarrow 2$
	DS2	$1 \Rightarrow 9$	$1 \Rightarrow 5$
Q18	A4S	$1 \Rightarrow 4$	$1 \Rightarrow 2$
	DS2	$1 \Rightarrow 8$	$1 \Rightarrow 4$
Q20	A4S	$1 \Rightarrow 7$	$1 \Rightarrow 2$
	DS2	$1 \Rightarrow 8$	$1 \Rightarrow 4$

Table 5.2: The output of A4S and DS2 scaling policies. *Parallelism* refers to the parallelism level of the bottleneck operator before and after scaling. *WorkerNum* refers to the number of workers (compute nodes) required for the query before and after scaling.

Query#	Parallelism	Memory Requirement (GB)
Q4	5	1.85
	6	1.58
	7	1.24
	8	0.90
	9	0.59
Q18	3	2.02
	4	1.71
	5	1.42
	6	1.12
	7	0.81
Q20	8	0.63
	5	2.94
	6	1.88
	7	1.41
	8	1.32

Table 5.3: Memory-parallelism curves generated by A4S.

operators remains one after scaling and is not shown in the table.

A4S considers the effect of memory on throughput and makes efficient scaling and placement decisions for the bottleneck operator based on the operator’s memory-parallelism curve, shown in Table 5.3 and Figure 5.3. For example, A4S chooses a parallelism level of 6 for Q4. The corresponding memory requirement for the bottleneck operator is 1.58 GB. These memory requirements fit within 2 workers since each worker has 850 MB of cache. A4S allocates vCores based on CPU usage (which indirectly accounts for the IO operations of the stateful operator), and it determines that the 4 vCores of the 2 workers are sufficient for the 6 tasks of the bottleneck operator.

DS2 decides the parallelism of the bottleneck operator $P_{bottleneck}$ by dividing the target throughput by the true rate (current throughput). This parallelism is higher than A4S’s output because it does not take the change in IO operations into account.

DS2’s Flink placement allocates slots based on the operator that has the maximum parallelism among the job’s operators. Thus, DS2’s scaling decisions require $\lceil P_{bottleneck}/2 \rceil$ workers, since each compute node has 2 vCores or slots. Assuming that operators were assigned parallelism based on A4S, Flink’s placement would still require more workers than the number of workers needed in A4S. This is because Flink’s placement doesn’t take CPU usage of tasks into account.

The top row of Figure 5.4 shows A4S and DS2 query throughput. These results show that the scal-

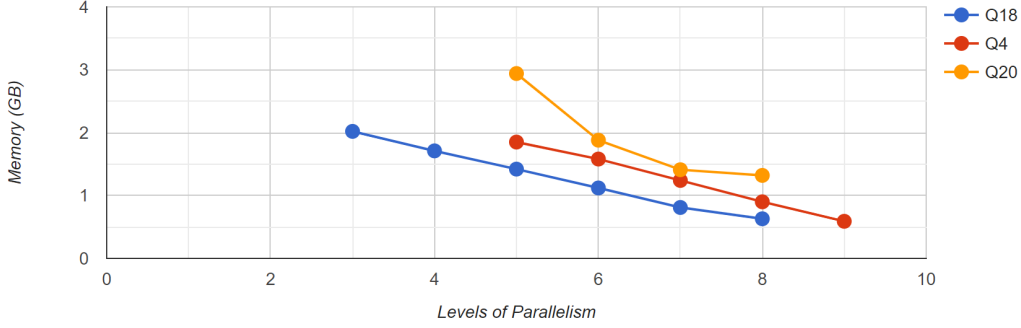


Figure 5.3: Memory-parallelism curves generated by A4S.

ing decisions of DS2 usually lead to over-provisioning. The achieved throughput is roughly $2\times$ the target throughput. In contrast, A4S generates more accurate scaling decisions, achieving a throughput close to the target throughput after scaling.

The middle row of Figure 5.4 shows A4S end-to-end query latency. This latency increases after scaling because the higher level of parallelism leads to straggler effects. The bottom row of Figure 5.4 shows DS2 end-to-end query latency. DS2 over-provisions and provides much more memory for the operators, so it has a lower miss rate and, thus, lower latency than A4S.

An unexpected observation from the top row of Figure 5.4 is that throughput gradually decreases after scaling, both with A4S and DS2. We discovered this occurs due to a misalignment when reading partitions from Kafka. After scaling, while we keep the parallelism of the source operator unchanged, the source tasks may be moved to other workers based on the placement decisions. These moved tasks usually require some warm-up time. As a result, existing unmoved source tasks consume their partitions from Kafka faster than moved source tasks. This results in the read offsets of the partitions being misaligned. When stateful operators such as join and aggregation access their corresponding keys, e.g., join keys, in different partitions, then the cache miss rate increases because the earlier arriving key may have been evicted by the time the later key is processed. This problem may persist and lower throughput over time since the processing latency of the later key is always higher than for the earlier key. Unlike the occasionally delayed records generated by our benchmark, this persistent processing skew across partitions violates our skew assumptions. This misalignment issue also occurs in other systems [7, 32], such as Flink [4] and Heron [42]. One solution is to control misalignment by regularly measuring the read offset of each partition. When a partition is far ahead, it is slowed down, which helps reduce the processing skew [5]. Puma [32] also explores this problem, and uses adaptive stream synchronization to control misalignment. We expect that these solutions can be applied in Risingwave as well and they will help reduce throughput degradation after scaling.

5.3 A4S Overhead

There are three potential causes of performance overhead in A4S: 1) the overhead of collecting and sending metrics, which is small since the metrics require little memory and are only reported once per second, 2) the overhead of maintaining LRU buckets and calculating stack distance, and 3) the keys in the ghost cache use extra memory, which reduces the memory that can be used for the LRU cache entries, possibly leading to more cache misses.

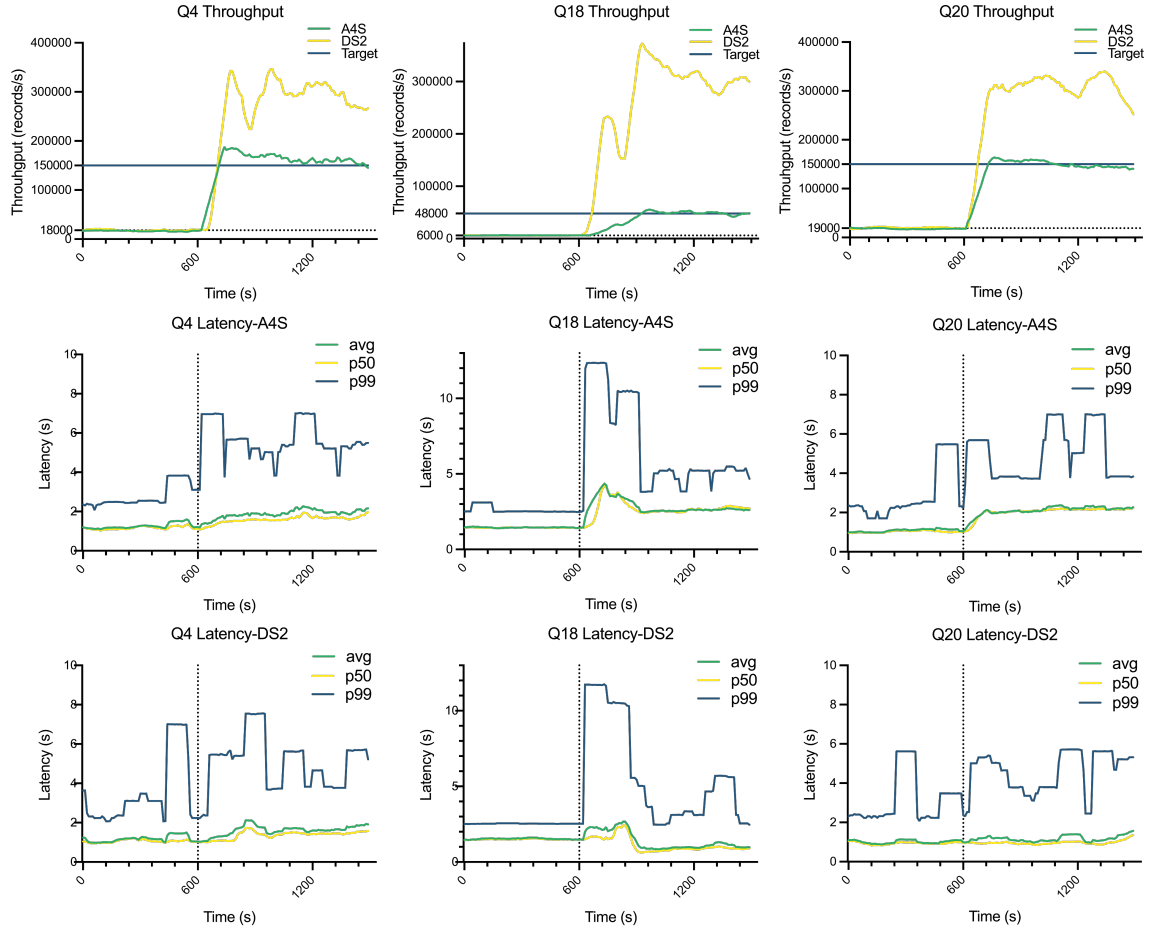


Figure 5.4: The top row shows query throughput before and after scaling with the DS2 and A4S scaling policy. The middle and bottom rows show the average, median, and P99 latency before and after scaling with A4S and DS2 respectively.

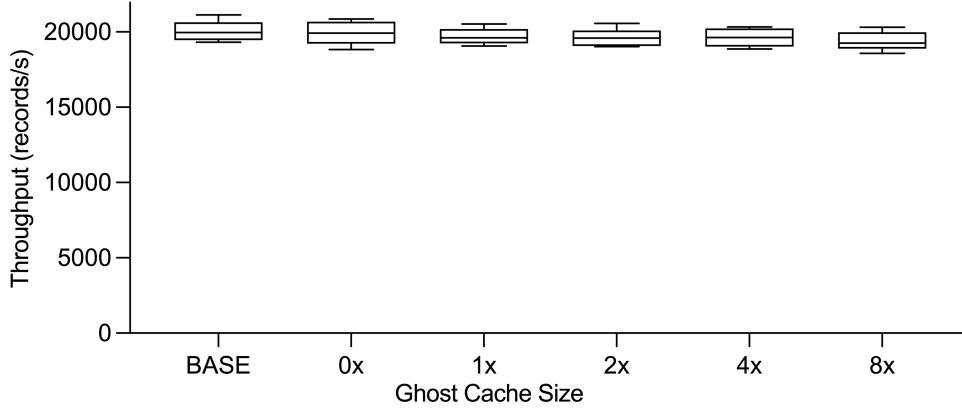


Figure 5.5: A4S throughput with different ghost cache sizes. For example, $2\times$ means the ghost cache has twice the number of items in the LRU cache.

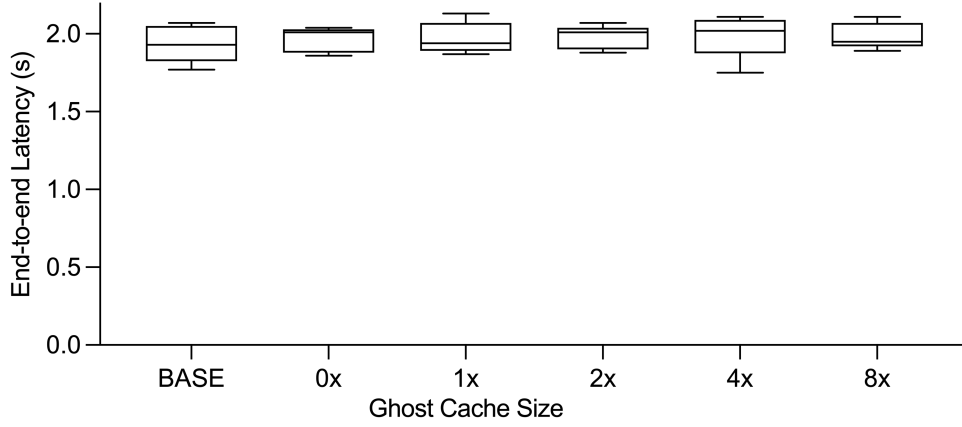


Figure 5.6: A4S latency with different ghost cache sizes.

To quantify these overheads, we measure the throughput and latency of A4S against baseline Risingwave v0.18.0.¹ For this experiment, we use the NEXMark Q4 query and deploy it with a parallelism level of one, like in Section 5.2, but we do not scale the query. For this experiment, we wait for the system to stabilize, and then we measure throughput and latency each second and average these measurements over a 30-minute interval. Even with this averaging, we find that the performance of Risingwave on the AWS EC2 instance varies across experimental runs. Thus, we run the experiment multiple times and report the average and deviation of the per-run average measurements.

Figure 5.5 and Figure 5.6 show the boxplots of throughput and latency for the baseline and for A4S with different ghost cache sizes. The number of keys in the ghost cache are $0\times$, $1\times$, $2\times$, $4\times$, and $8\times$ the keys in the LRU cache. With $0\times$, there is no ghost cache, but we still use LRU buckets and sample stack distances in the LRU cache.

Overall, we see that the throughput and latency overheads are small. The overhead of $0\times$ ghost cache size over the baseline is insignificant, showing that the overhead of LRU buckets and sampling of stack distances is negligible. With $2\times$ ghost cache size, the average throughput decreases by 2.2%, and the average latency

¹The baseline uses the frontend modification mentioned in Section 4.

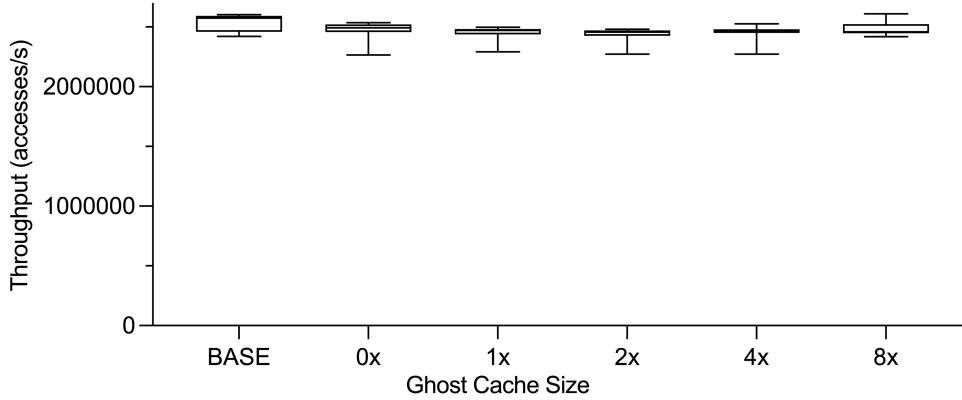


Figure 5.7: Throughput of an LRU cache using Quickmrc with different ghost cache sizes.

increases by 2.1% compared to the baseline. At $8\times$ ghost cache size, the corresponding numbers are 3.4% and 2.7%. Comparing $8\times$ and $0\times$, the overhead of the ghost cache is also limited (3.1% for average throughput and 1.2% for average latency). These performance overheads are acceptable compared to the benefit of A4S scaling. We expect they can be further reduced if we only keep sampled keys in the ghost cache.

5.4 Performance and Accuracy of Quickmrc

Besides cache operations, streaming processing contains several compute-bound and IO operations. Thus, we use a microbenchmark to assess the performance of the Quickmrc method. We create an access trace of 10M consecutive access keys for the join operator in the NEXMark Q4 query, using our modified NEXMark benchmark (i.e., 2% of the records may be delayed). The microbenchmark only contains LRU cache operations (lookup and put) and miss-rate curve related operations. We use an LRU cache with 50000 entries to analyze the trace. For Quickmrc, we set the number of LRU buckets to 50, the sampling rate to 3%, and the number of histogram buckets to 50. We use 4 KB for the value size of a cache item, which is close to the item size in NEXMark Q4), so the cache size is roughly 200 MB.

Figure 5.7 shows the throughput of an LRU cache that uses Quickmrc with different ghost cache sizes. Compared to the baseline LRU cache, Quickmrc has roughly 4% overhead without a ghost cache and a maximum of 8% overhead with a ghost cache. This overhead is small when compared to the overall cost of streaming operations, as shown in Figures 5.5 and 5.6.

For the same experiment, Figure 5.8 shows the accurate (Mattson) miss-rate curve and the miss-rate curves for three versions of Quickmrc, without a ghost cache. Quickmrc introduces inaccuracy due to LRU buckets, sampling, and histogram buckets. The figure shows the miss-rate curve when using LRU buckets alone, then sampling the LRU buckets, and finally, the Quickmrc curve, which uses histogram buckets to generate miss-rate curves. For the accurate, LRU bucket and sampling miss-rate curves, we bin the stack distance into 1000 fixed-size buckets of size 200 KB (50 items). For the histogram bucket miss-rate curve, we bin the stack distance in 50 buckets.

The exact miss rate drops sharply as the cache size increases when the cache size is small because a streaming workload usually has a good locality. To quantify the accuracy of the three versions, we measure the Mean Absolute Error (MAE) [66]. The absolute error is the absolute difference between the estimated

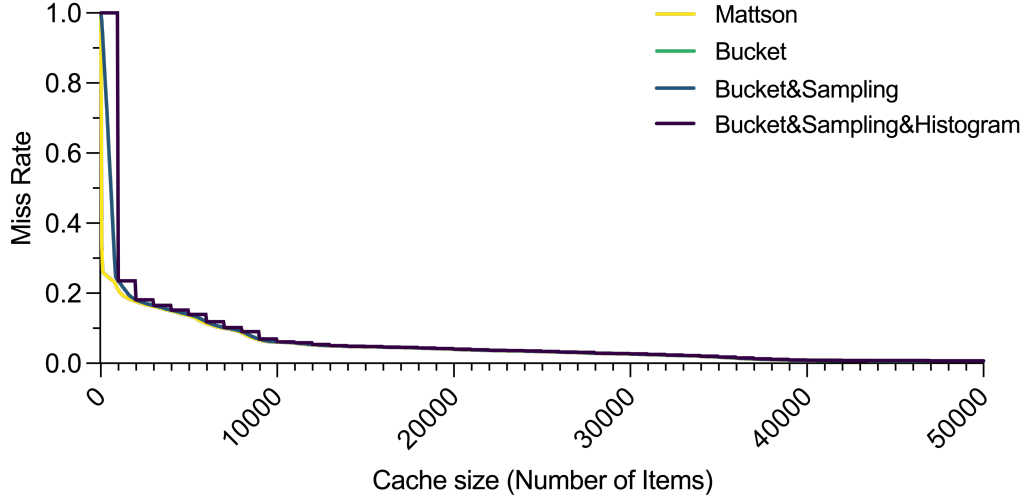


Figure 5.8: Miss rate curves with LRU buckets, sampling and histogram buckets in Quickmrc.

and the actual miss rate for a given cache size. Since the miss rate lies between 0 and 1, the absolute error also lies between 0 and 1 (or 100%). We calculate MAE by averaging the absolute error for the 1000 discrete cache sizes of size 200 KB (50 items).

The MAE of the LRU bucket, sampling, and histogram versions is 0.68%, 0.68%, and 1.65%, respectively. With 50 LRU buckets, the worst case error is 2%, and the average error is 1%, but the bucket sizes are variable and shrink as they age, which reduces the average error further. The sampling miss rate curve overlaps with the LRU bucket curve in Figure 5.8, and their MAE are close, so sampling at 3% rate introduces minimal additional inaccuracy. The histogram miss-rate curve shows a staircase shape. With 50 histogram buckets, the additional error in the worst case is 2%, and the average case is 1%. Combining this error with the LRU bucket and sampling MAE error yields close to the expected MAE error.

We see that the inaccuracy of Quickmrc is much higher at small cache sizes. In practice, the stateful operators in A4S require significant cache memory to reduce IO operations, so their miss rate is low, and Quickmrc has reasonable accuracy in that miss-rate range.

Chapter 6

Conclusions

This thesis presents A4S, a novel co-designed scaling and placement strategy for stream processing systems that works well for both stateless and stateful operators. A4S captures the correlation between operator memory requirements and operator parallelism and formalizes this correlation with a memory-parallelism curve, which shows the minimum operator cache memory size needed for a given level of parallelism to achieve a target throughput. Memory-parallelism curves enable trading memory and parallelism, and this flexibility enables making more efficient scaling and placement decisions.

A4S co-designs scaling and placement with a greedy strategy that initially allocates a minimal number of hardware nodes that can satisfy a jobs' target throughput. Then, it places operators on these nodes using their memory-parallelism curves until either their memory and CPU demands can be met, or else it increases hardware nodes, and then retries placement.

We show that memory-parallelism curves for an operator can be derived from the miss-rate curve of the operator cache. Generating miss-rate curves requires determining the stack distance of cache accesses. We design Quickmrc, an online miss-rate curve generation scheme suited for streaming systems. Quickmrc uses cache buckets and spatial sampling to reduce the cost of stack distance calculation and it uses histogram buckets to pre-aggregate stack distances, thus enabling efficient generation of miss-rate curves.

We implement A4S on the Risingwave streaming system. Our evaluation shows that A4S can provide more accurate and resource-efficient scaling and placement decisions for memory-intensive stateful operators compared to the state-of-the-art DS4 system. We also show that A4S imposes low performance overhead. Using microbenchmarks, we also show that Quickmrc has low overhead and the inaccuracy introduced by cache buckets, sampling, and histogram buckets are acceptable for streaming systems.

Chapter 7

Future Work

This thesis exposes several interesting directions for future research.

Sensitivity analysis for locality The 2% random delay and the corresponding 2% cache miss used in our experiments are based on a production Risingwave cluster. However, it will be useful to evaluate A4S performance for different record delays and cache miss rates to fully understand the benefits of A4S.

Applications of QuickMRC We have shown that Quickmrc has low overhead and reasonable accuracy for streaming applications. We expect that other applications with similar requirements will also benefit from Quickmrc.

Comparison with other miss-rate algorithms It will be useful to compare the performance and accuracy of QuickMRC with Mimir and SHARDS, our baseline miss-rate schemes.

Multi-cache operator support Currently, A4S only supports the join operator with multiple caches. It should be possible to support other operators that use multiple caches using a design similar to the join operator, which would generalize the A4S approach.

Sampling in LRU ghost cache The current LRU implementation keeps all the evicted keys in the ghost cache to determine the stack distance beyond the LRU cache size. However, similar to SHARDS [66], only the keys that are sampled for calculating the stack distance need to be stored in the ghost cache. This may cause some additional inaccuracy but it will reduce the memory and performance overhead of Quickmrc further.

Handling misalignment in Risingwave Currently, Risingwave does not handle misalignment between the partitions of the upstream systems or between two streams, which violates the low-skew assumption in A4S. This misalignment leads to performance degradation, so it is important to address this issue.

Auto-scaling While A4S provides an accurate scaling strategy, it still requires the user to specify the target throughput of source operators and the maximum latency of stateful operators. An ideal scaling solution

would automatically determine the target throughput based on the throughput and the amount of data accumulated in the upstream system and deduce the maximum operator latency based on the end-to-end latency SLO.

Handling data skew A4S assumes low data skew across partitions. However, real-world data distributions are complicated and may be highly skewed. Some systems are designed to handle data skew while scaling. It will be interesting to explore the feasibility of combining A4S and these works for streaming systems.

Handling load bursts and long-term load changes A4S assumes the workload has relatively stable characteristics. However, load bursts are common in production systems. Handling bursty loads and long-term throughput changes together within a single streaming system is also an interesting future research direction.

Applicability of memory-parallelism curves We have explored the use of memory-parallelism curves in streaming systems. We plan to explore the use of memory-parallelism curves in other systems that have memory-intensive workloads and long-running computations scaled using parallelizable operations, e.g., batch processing systems, graph processing systems, etc. Similar to this work, we expect that memory-parallelism curves can provide a range of choices for memory allocation, parallelism and placement in such systems.

Bibliography

- [1] AWS Kinesis. <https://aws.amazon.com/kinesis/>.
- [2] AWS S3. <https://aws.amazon.com/s3/>.
- [3] Azure Blob Storage. <https://azure.microsoft.com/en-us/products/storage/blobs/>.
- [4] Misalignment Issue in Flink. <https://issues.apache.org/jira/browse/FLINK-28853>.
- [5] Misalignment Solution in Flink. <https://github.com/apache/flink/pull/20485>.
- [6] Nexmark Benchmark Suite of Flink. <https://github.com/nexmark/nexmark>.
- [7] Daniel J Abadi, Don Carney, Ugur Cetintemel, Mitch Cherniack, Christian Convey, Sangdon Lee, Michael Stonebraker, Nesime Tatbul, and Stan Zdonik. Aurora: a new model and architecture for data stream management. *the VLDB Journal*, 12:120–139, 2003.
- [8] Tyler Akidau, Alex Balikov, Kaya Bekiroglu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. Millwheel: Fault-tolerant stream processing at internet scale. In *Very Large Data Bases*, pages 734–746, 2013.
- [9] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. 2015.
- [10] George Almási, Călin Caşcaval, and David A Padua. Calculating stack distances efficiently. In *Proceedings of the 2002 workshop on Memory system performance*, pages 37–43, 2002.
- [11] Ganesh Ananthanarayanan, Ali Ghodsi, Scott Shenker, and Ion Stoica. Effective straggler mitigation: Attack of the clones. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 185–198, 2013.
- [12] Bryan T Bennett and Vincent J. Kruskal. Lru stack processing. *IBM Journal of Research and Development*, 19(4):353–357, 1975.
- [13] Erik Berg and Erik Hagersten. Statcache: A probabilistic approach to efficient and accurate data locality analysis. In *IEEE International Symposium on-ISPASS Performance Analysis of Systems and Software, 2004*, pages 20–27. IEEE, 2004.

- [14] Erik Berg and Erik Hagersten. Fast data-locality profiling of native execution. In *Proceedings of the 2005 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*, pages 169–180, 2005.
- [15] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.
- [16] Valeria Cardellini, Vincenzo Grassi, Francesco Lo Presti, and Matteo Nardelli. Optimal operator placement for distributed stream processing applications. In *Proceedings of the 10th ACM International Conference on Distributed and Event-based Systems*, pages 69–80, 2016.
- [17] Valeria Cardellini, Matteo Nardelli, and Dario Luzi. Elastic stateful stream processing in storm. In *2016 International Conference on High Performance Computing & Simulation (HPCS)*, pages 583–590. IEEE, 2016.
- [18] Damiano Carra and Giovanni Neglia. Efficient miss ratio curve computation for heterogeneous content popularity. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 741–751, 2020.
- [19] Raul Castro Fernandez, Matteo Migliavacca, Evangelia Kalyvianaki, and Peter Pietzuch. Integrating scale out and fault tolerance in stream processing using operator state management. In *Proceedings of the 2013 ACM SIGMOD international conference on Management of data*, pages 725–736, 2013.
- [20] Javier Cervino, Evangelia Kalyvianaki, Joaquin Salvachua, and Peter Pietzuch. Adaptive provisioning of stream processing systems in the cloud. In *2012 IEEE 28th International Conference on Data Engineering Workshops*, pages 295–301. IEEE, 2012.
- [21] Asaf Cidon, Assaf Eisenman, Mohammad Alizadeh, and Sachin Katti. Cliffhanger: Scaling performance cliffs in web memory caches. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 379–392, 2016.
- [22] Tiziano De Matteis and Gabriele Mencagli. Elastic scaling for distributed latency-sensitive data stream operators. In *2017 25th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*, pages 61–68. IEEE, 2017.
- [23] Bonaventura Del Monte, Steffen Zeuch, Tilmann Rabl, and Volker Markl. Rhino: Efficient management of very large distributed state for stream processing engines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2471–2486, 2020.
- [24] Omar Farhat, Khuzaima Daudjee, and Leonardo Querzoni. Klink: Progress-aware scheduling for streaming data systems. In *Proceedings of the 2021 International Conference on Management of Data*, pages 485–498, 2021.
- [25] Avrilia Floratou, Ashvin Agrawal, Bill Graham, Sriram Rao, and Karthikeyan Ramasamy. Dhalion: Self-regulating stream processing in heron. *Proc. VLDB Endow.*, 10:1825–1836, 2017.
- [26] Buğra Gedik, Scott Schneider, Martin Hirzel, and Kun-Lung Wu. Elastic scaling for data stream processing. *IEEE Transactions on Parallel and Distributed Systems*, 25(6):1447–1463, 2013.

- [27] Buğra Gedik, Scott Schneider, Martin Hirzel, and Kun-Lung Wu. Elastic scaling for data stream processing. *IEEE Transactions on Parallel and Distributed Systems*, 25(6):1447–1463, 2013.
- [28] Philipp M Grulich, Breß Sebastian, Steffen Zeuch, Jonas Traub, Janis von Bleichert, Zongxiong Chen, Tilmann Rabl, and Volker Markl. Grizzly: Efficient stream processing through adaptive query compilation. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2487–2503, 2020.
- [29] Rong Gu, Han Yin, Weichang Zhong, Chunfeng Yuan, and Yihua Huang. Mecas: Latency-efficient rescaling via prioritized state migration for stateful distributed stream processing systems. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 539–556, 2022.
- [30] Vincenzo Gulisano, Ricardo Jimenez-Peris, Marta Patino-Martinez, Claudio Soriente, and Patrick Valduriez. Streamcloud: An elastic and scalable data streaming system. *IEEE Transactions on Parallel and Distributed Systems*, 23(12):2351–2365, 2012.
- [31] Moritz Hoffmann, Andrea Lattuada, Frank McSherry, Vasiliki Kalavri, John Liagouris, and Timothy Roscoe. Megaphone: Latency-conscious state migration for distributed streaming dataflows. *Proceedings of the VLDB Endowment*, 12(9):1002–1015, 2019.
- [32] Gabriela Jacques-Silva, Ran Lei, Luwei Cheng, Guoqiang Jerry Chen, Kuen Ching, Tanji Hu, Yuan Mei, Kevin Wilfong, Rithin Shetty, Serhat Yilmaz, et al. Providing streaming joins as a service at facebook. *Proceedings of the VLDB Endowment*, 11(12):1809–1821, 2018.
- [33] Anand Jayarajan, Kimberly Hau, Andrew Goodwin, and Gennady Pekhimenko. Lifestream: a high-performance stream processing engine for periodic streams. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 107–122, 2021.
- [34] Anand Jayarajan, Wei Zhao, Yudi Sun, and Gennady Pekhimenko. Tilt: A time-centric approach for stream query optimization and parallelization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 818–832, 2023.
- [35] Albert Jonathan, Abhishek Chandra, and Jon Weissman. Wasp: Wide-area adaptive stream processing. In *Proceedings of the 21st International Middleware Conference*, pages 221–235, 2020.
- [36] Vasiliki Kalavri, John Liagouris, Moritz Hoffmann, Desislava Dimitrova, Matthew Forshaw, and Timothy Roscoe. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 783–798, Carlsbad, CA, October 2018. USENIX Association.
- [37] Faria Kalim, Le Xu, Sharanya Bathey, Richa Meherwal, and Indranil Gupta. Henge: Intent-driven multi-tenant stream processing. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 249–262, 2018.
- [38] Yul H Kim, Mark D Hill, and David A Wood. Implementing stack simulation for highly-associative memories. *ACM SIGMETRICS Performance Evaluation Review*, 19(1):212–213, 1991.

- [39] Wilhelm Kleiminger, Evangelia Kalyvianaki, and Peter Pietzuch. Balancing load in stream processing with the cloud. In *2011 IEEE 27th International Conference on Data Engineering Workshops*, pages 16–21. IEEE, 2011.
- [40] Alexandros Kolios, Matthias Weidlich, Raul Castro Fernandez, Alexander L Wolf, Paolo Costa, and Peter Pietzuch. Saber: Window-based hybrid stream processing for heterogeneous architectures. In *Proceedings of the 2016 International Conference on Management of Data*, pages 555–569, 2016.
- [41] Lingkun Kong and Konstantinos Mamouras. Streamql: a query language for processing streaming time series. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–32, 2020.
- [42] Sanjeev Kulkarni, Nikunj Bhagat, Maosong Fu, Vikas Kedigehalli, Christopher Kellogg, Sailesh Mittal, Jignesh M Patel, Karthik Ramasamy, and Siddarth Taneja. Twitter heron: Stream processing at scale. In *Proceedings of the 2015 ACM SIGMOD international conference on Management of data*, pages 239–250, 2015.
- [43] Jin Li, Kristin Tufte, Vladislav Shkapenyuk, Vassilis Papadimos, Theodore Johnson, and David Maier. Out-of-order processing: a new architecture for high-performance stream systems. *Proceedings of the VLDB Endowment*, 1(1):274–288, 2008.
- [44] Shen Li, Paul Gerver, John MacMillan, Daniel Debrunner, William Marshall, and Kun-Lung Wu. Challenges and experiences in building an efficient apache beam runner for ibm streams. *Proceedings of the VLDB Endowment*, 11(12):1742–1754, 2018.
- [45] John DC Little. A proof for the queuing formula: $L = \lambda w$. *Operations research*, 9(3):383–387, 1961.
- [46] Xunyun Liu and Rajkumar Buyya. Performance-oriented deployment of streaming applications on cloud. *IEEE Transactions on Big Data*, 5(1):46–59, 2017.
- [47] Zhang Liu, Hee Won Lee, Yu Xiang, Dirk Grunwald, and Sangtae Ha. eMRC: Efficient miss ratio approximation for Multi-Tier caching. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 293–306, February 2021.
- [48] Luo Mai, Kai Zeng, Rahul Potharaju, Le Xu, Steve Suh, Shivaram Venkataraman, Paolo Costa, Terry Kim, Saravanan Muthukrishnan, Vamsi Kuppa, et al. Chi: A scalable and programmable control plane for distributed stream processing systems. *Proceedings of the VLDB Endowment*, 11(10):1303–1316, 2018.
- [49] Richard L. Mattson, Jan Gecsei, Donald R. Slutz, and Irving L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems journal*, 9(2):78–117, 1970.
- [50] Frank McSherry, Andrea Lattuada, Malte Schwarzkopf, and Timothy Roscoe. Shared arrangements: practical inter-query sharing for streaming dataflows. *arXiv preprint arXiv:1812.02639*, 2018.
- [51] Hongyu Miao, Myeongjae Jeon, Gennady Pekhimenko, Kathryn S McKinley, and Felix Xiaozhu Lin. Streambox-hbm: Stream analytics on high bandwidth hybrid memory. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 167–181, 2019.

- [52] Jorge R Murillo, Gustavo Totoy, and Cristina L Abad. Instrumenting cloud caches for online workload monitoring: The case of online miss rate curve estimation in memcached. In *Proceedings of the 16th Workshop on Adaptive and Reflective Middleware*, pages 1–6, 2017.
- [53] Muhammad Anis Uddin Nasir, Gianmarco De Francisci Morales, David Garcia-Soriano, Nicolas Kourtellis, and Marco Serafini. The power of both choices: Practical load balancing for distributed stream processing engines. In *2015 IEEE 31st International Conference on Data Engineering*, pages 137–148. IEEE, 2015.
- [54] Qingpeng Niu, James Dinan, Qingda Lu, and Ponnuswamy Sadayappan. Parda: A fast parallel reuse distance analysis algorithm. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium*, pages 1284–1294. IEEE, 2012.
- [55] Frank Olken. Efficient methods for calculating the success function of fixed-space replacement policies. Technical report, Lawrence Berkeley National Lab.(LBNL), Berkeley, CA (United States), 1981.
- [56] Nicolás Rivetti, Emmanuelle Anceaume, Yann Busnel, Leonardo Querzoni, and Bruno Sericola. Online scheduling for shuffle grouping in distributed stream processing systems. In *Proceedings of the 17th International Middleware Conference*, pages 1–12, 2016.
- [57] Stamatia Rizou, Frank Durr, and Kurt Rothermel. Solving the multi-operator placement problem in large-scale operator networks. In *2010 Proceedings of 19th International Conference on Computer Communications and Networks*, 2010.
- [58] Trausti Saemundsson, Hjortur Bjornsson, Gregory Chockler, and Ymir Vigfusson. Dynamic performance profiling of cloud caches. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 1–14, 2014.
- [59] Trausti Saemundsson, Hjortur Bjornsson, Gregory Chockler, and Ymir Vigfusson. Dynamic performance profiling of cloud caches. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 1–14, 2014.
- [60] Derek L Schuff, Milind Kulkarni, and Vijay S Pai. Accelerating multicore reuse distance analysis with sampling and parallelization. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, pages 53–64, 2010.
- [61] Won Wook Song, Taegeon Um, Sameh Elnikety, Myeongjae Jeon, and Byung-Gon Chun. Sponge: Fast reactive scaling for stream processing with serverless frameworks. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 301–314, 2023.
- [62] Utkarsh Srivastava and Jennifer Widom. Flexible time management in data stream systems. In *Proceedings of the twenty-third ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 263–274, 2004.
- [63] David K Tam, Reza Azimi, Livio B Soares, and Michael Stumm. Rapidmrc: approximating l2 miss rate curves on commodity systems for online optimizations. *ACM Sigplan Notices*, 44(3):121–132, 2009.
- [64] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. Nexmark—a benchmark for queries over data streams (draft). *Technical report*, 2008.

- [65] Midhul Vuppapapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. Building an elastic query engine on disaggregated storage. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 449–462, 2020.
- [66] Carl A Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient {MRC} construction with {SHARDS}. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 95–110, 2015.
- [67] Guozhang Wang, Lei Chen, Ayusman Dikshit, Jason Gustafson, Boyang Chen, Matthias J Sax, John Roesler, Sophie Blee-Goldman, Bruno Cadonna, Apurva Mehta, et al. Consistency and completeness: Rethinking distributed stream processing in apache kafka. In *Proceedings of the 2021 international conference on management of data*, pages 2602–2613, 2021.
- [68] Yanghao Wang and Zhi Liu. A sneak peek at risingwave: a cloud-native streaming database. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*, pages 190–193, 2022.
- [69] Xiaohui Wei, Lina Li, Xiang Li, Xingwang Wang, Shang Gao, and Hongliang Li. Pec: Proactive elastic collaborative resource scheduling in data stream processing. *IEEE Transactions on Parallel and Distributed Systems*, 30(7):1628–1642, 2019.
- [70] Jake Wires, Stephen Ingram, Zachary Drudi, Nicholas JA Harvey, and Andrew Warfield. Characterizing storage workloads with counter stacks. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 335–349, 2014.
- [71] Yingjun Wu and Kian-Lee Tan. Chronostream: Elastic stateful stream computation in the cloud. In *2015 IEEE 31st International Conference on Data Engineering*, pages 723–734. IEEE, 2015.
- [72] Junyao Yang, Yuchen Wang, and Zhenlin Wang. Efficient modeling of random sampling-based lru. In *Proceedings of the 50th International Conference on Parallel Processing*, pages 1–11, 2021.
- [73] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*, pages 423–438, 2013.
- [74] Fan Zhang, Hanhua Chen, and Hai Jin. Simois: A scalable distributed stream join system with skewed workloads. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pages 176–185. IEEE, 2019.
- [75] Weiming Zhao, Xinxin Jin, Zhenlin Wang, Xiaolin Wang, Yingwei Luo, and Xiaoming Li. Low cost working set size tracking. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*, 2011.
- [76] Yutao Zhong and Wentao Chang. Sampling-based program locality approximation. In *Proceedings of the 7th international symposium on Memory management*, pages 91–100, 2008.